



Lecture #10

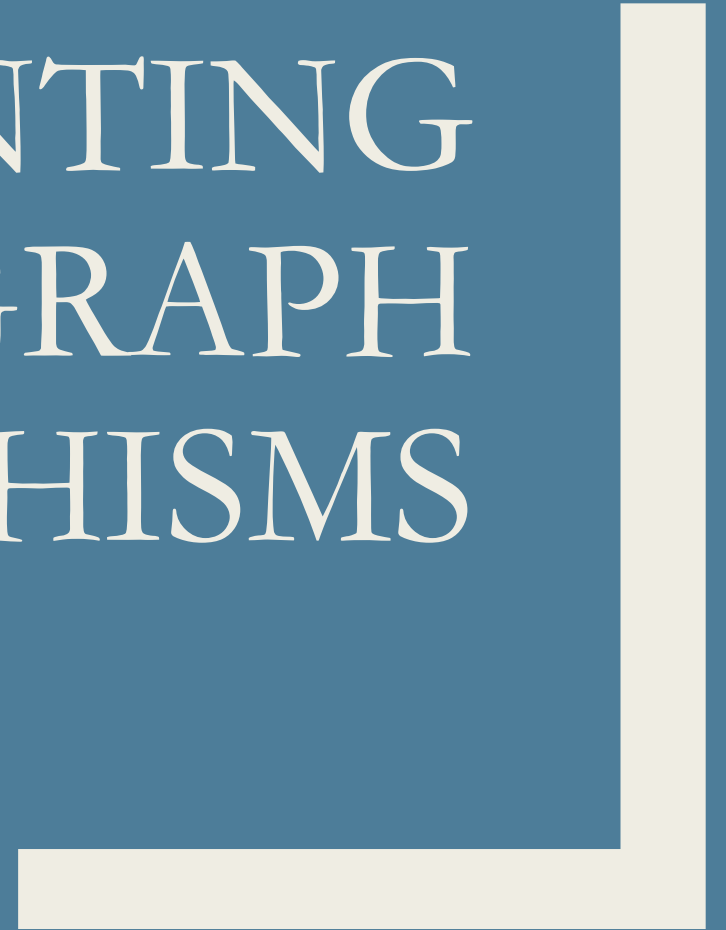
GRAPH THEORY

Connectivity, Trees, and Graph Isomorphism

Senior lecturer,
Aknur Askarbekkyzy



REPRESENTING GRAPHS AND GRAPH ISOMORPHISMS



Representing Graphs

There are many useful ways to represent graphs.

One way to represent a graph is to use **adjacency lists**, which specify the vertices that are adjacent to each vertex of the graph.

Carrying out graph algorithms using the representation of graphs by adjacency lists, can be cumbersome if there are many edges in the graph. To simplify computation, graphs can be represented using **matrices**.

Adjacency list: Example

EXAMPLE. Use adjacency lists to describe the simple graph

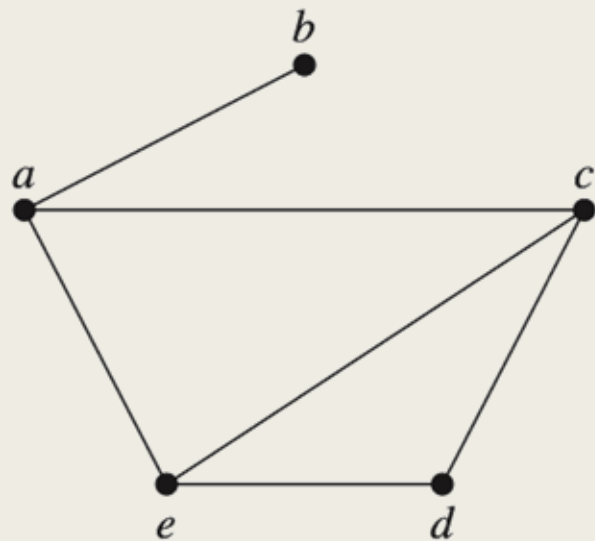


FIGURE 1 A Simple Graph.

TABLE 1 An Adjacency List for a Simple Graph.

<i>Vertex</i>	<i>Adjacent Vertices</i>
<i>a</i>	<i>b, c, e</i>
<i>b</i>	<i>a</i>
<i>c</i>	<i>a, d, e</i>
<i>d</i>	<i>c, e</i>
<i>e</i>	<i>a, c, d</i>

Adjacency list: Example

EXAMPLE. Represent the given directed graph by listing all the vertices that are the terminal vertices of edges starting at each vertex of the graph.

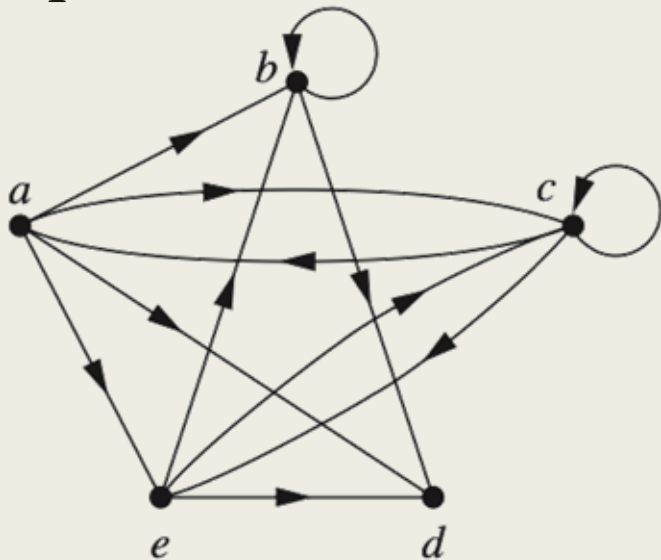


FIGURE 2 A Directed Graph.

TABLE 2 An Adjacency List for a Directed Graph.

<i>Initial Vertex</i>	<i>Terminal Vertices</i>
<i>a</i>	<i>b, c, d, e</i>
<i>b</i>	<i>b, d</i>
<i>c</i>	<i>a, c, e</i>
<i>d</i>	
<i>e</i>	<i>b, c, d</i>

Adjacency Matrices

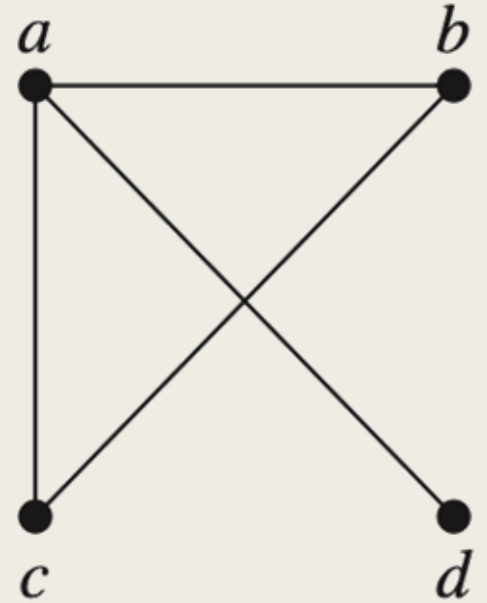
Two types of matrices commonly used to represent graphs will be presented here. One is based on the *adjacency* of vertices, and the other is based on *incidence* of vertices and edges.

Suppose that $G = (V, E)$ is a simple graph where $|V| = n$. Suppose that the vertices of G are listed arbitrarily as $v_1, v_2, \dots, v_n$. The ***adjacency matrix*** $\mathbf{A}$ (or $\mathbf{A}_G$) of G , with respect to this listing of the vertices, is the $n \times n$ zero–one matrix with 1 as its (i, j) -th entry when v_i and v_j are adjacent, and 0 as its (i, j) -th entry when they are not adjacent. In other words, if its adjacency matrix is $\mathbf{A} = [a_{ij}]$, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

Adjacency Matrices: Example

EXAMPLE. Use an adjacency matrix to represent the graph



Solution: We order the vertices as a, b, c, d. The matrix representing this graph is

$$\begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Adjacency Matrices

Note that an adjacency matrix of a graph is based on the ordering chosen for the vertices. Hence, there may be as many as $n!$ different adjacency matrices for a graph with n vertices.

The adjacency matrix of a simple graph is symmetric, that is, $a_{ij} = a_{ji}$.

Furthermore, because a simple graph has no loops, each entry a_{ii} , $i = 1, 2, 3, \dots, n$, is 0.

Adjacency Matrices

Adjacency matrices can also be used to represent undirected graphs with loops and with multiple edges.

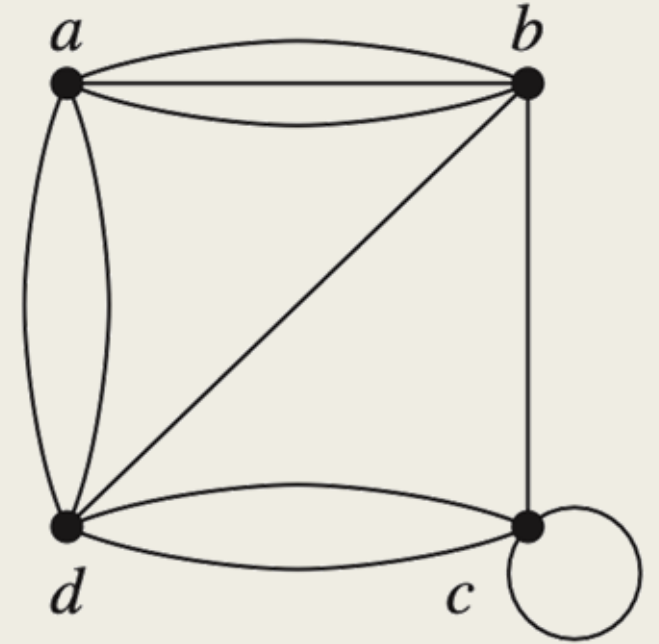
- A loop at the vertex v_i is represented by a 1 at the (i, i) -th position of the adjacency matrix.
- When multiple edges connecting the same pair of vertices v_i and v_j , or multiple loops at the same vertex, are present, the adjacency matrix is no longer a zero–one matrix, because the (i, j) -th entry of this matrix equals the number of edges that are associated to $\{v_i, v_j\}$.
- All undirected graphs, including multigraphs and pseudographs, have symmetric adjacency matrices.

Adjacency Matrices: Example

EXAMPLE. Use an adjacency matrix to represent the pseudograph

Solution: The adjacency matrix using the ordering of vertices a, b, c, d is

$$\begin{bmatrix} 0 & 3 & 0 & 2 \\ 3 & 0 & 1 & 1 \\ 0 & 1 & 1 & 2 \\ 2 & 1 & 2 & 0 \end{bmatrix}$$



Adjacency Matrices: Example

The matrix for a directed graph $G = (V, E)$ has a 1 in its (i, j) -th position if there is an edge from v_i to v_j , where $v_1, v_2, \dots, v_n$ is an arbitrary listing of the vertices of the directed graph. In other words, if $\mathbf{A} = [a_{ij}]$ is the adjacency matrix for the directed graph with respect to this listing of the vertices, then

$$a_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ 0 & \text{otherwise.} \end{cases}$$

NOTE: The adjacency matrix for a directed graph does not have to be symmetric, because there may not be an edge from v_j to v_i when there is an edge from v_i to v_j .

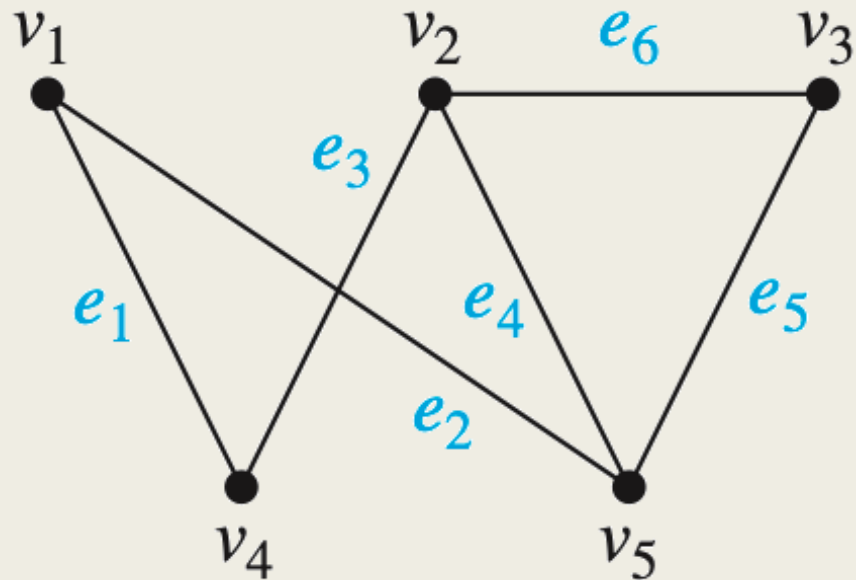
Incidence Matrices

Let $G = (V, E)$ be an undirected graph. Suppose that $v_1, v_2, \dots, v_n$ are the vertices and $e_1, e_2, \dots, e_m$ are the edges of G . Then the incidence matrix with respect to this ordering of V and E is the $n \times m$ matrix $\mathbf{M} = [m_{ij}]$, where

$$m_{ij} = \begin{cases} 1 & \text{when edge } e_j \text{ is incident with } v_i, \\ 0 & \text{otherwise.} \end{cases}$$

Incidence Matrices: Example

EXAMPLE. Represent the following graph with an incidence matrix.



$$\begin{array}{c} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{bmatrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

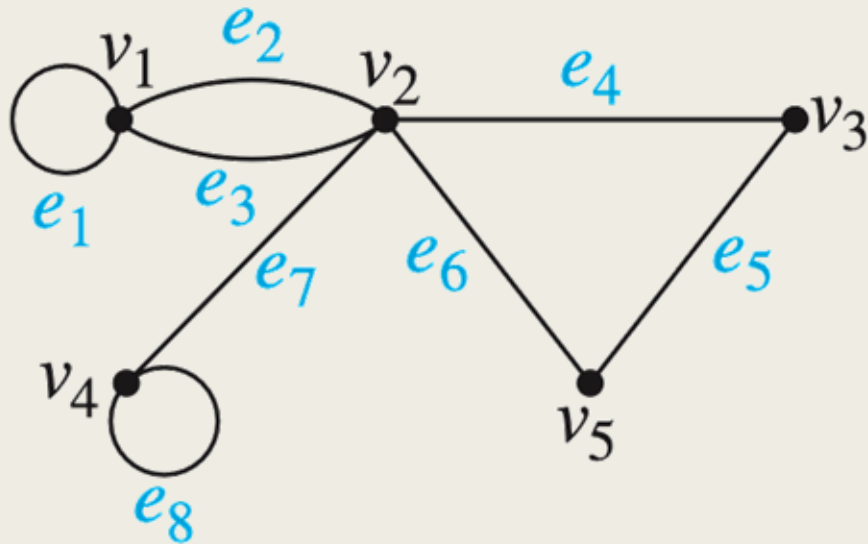
Incidence Matrices

Incidence matrices can also be used to represent multiple edges and loops.

- Multiple edges are represented in the incidence matrix using columns with identical entries, because these edges are incident with the same pair of vertices.
- Loops are represented using a column with exactly one entry equal to 1, corresponding to the vertex that is incident with this loop.

Incidence Matrices: Example

EXAMPLE. Represent the following pseudograph using an incidence matrix.



$$\begin{array}{c} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{array} \begin{bmatrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 & e_7 & e_8 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \end{bmatrix}$$

Isomorphism of Graphs

Sometimes, two graphs have exactly the same form, in the sense that there is a one-to-one correspondence between their vertex sets that preserves edges. In such a case, we say that the two graphs are *isomorphic*.

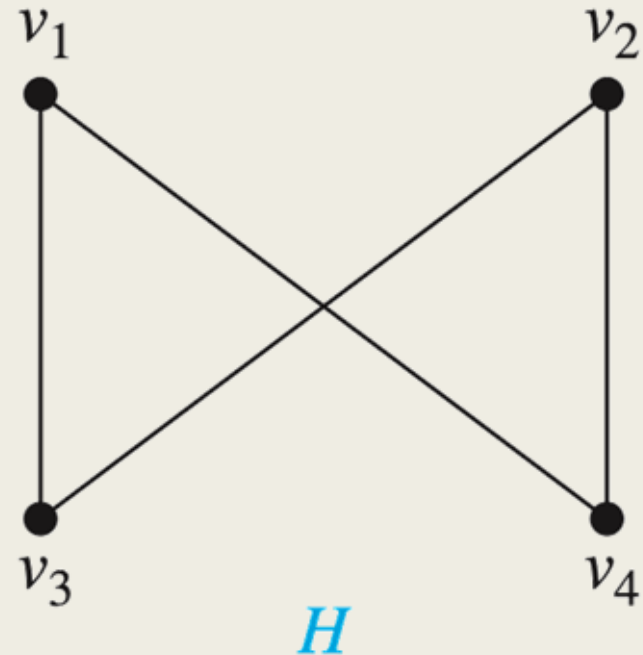
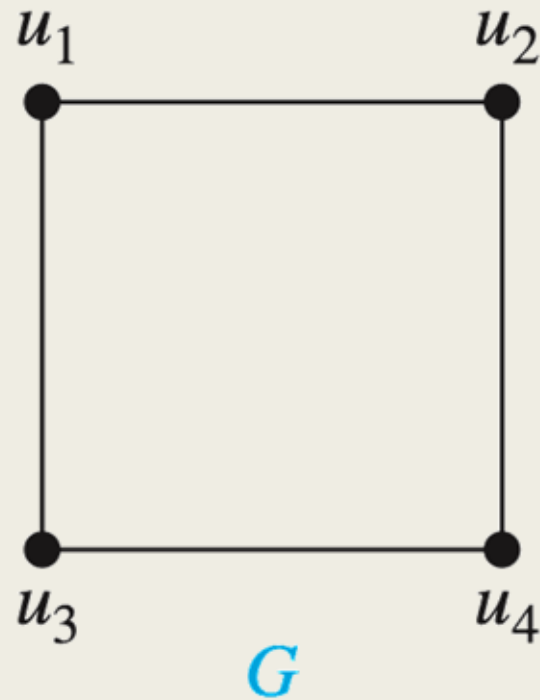
Isomorphism of Graphs

DEFINITION. The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are *isomorphic* if there exists a one-to-one and onto function f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 , for all a and b in V_1 . Such a function f is called an *isomorphism*.

Two simple graphs that are not isomorphic are called *nonisomorphic*.

Isomorphism of Graphs: Example

EXAMPLE. Show that the graphs $G = (V, E)$ and $H = (W, F)$ are isomorphic.



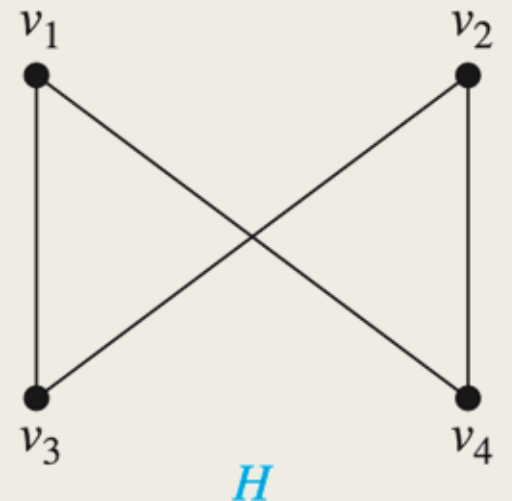
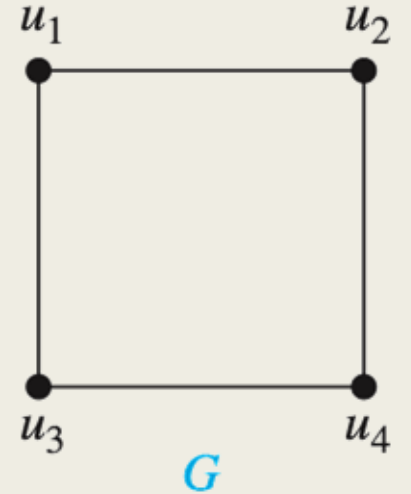
Isomorphism of Graphs: Example

Solution: The function f with

$$f(u_1) = v_1, \quad f(u_2) = v_4,$$

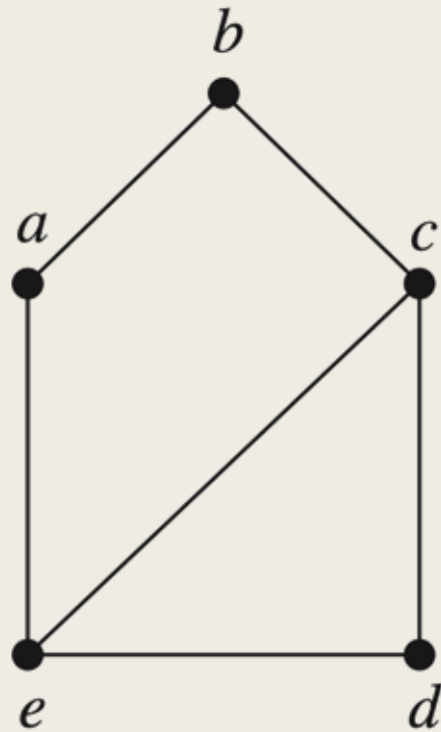
$$f(u_3) = v_3, \text{ and } f(u_4) = v_2$$

is a one-to-one correspondence between V and W . To see that this correspondence preserves adjacency, note that adjacent vertices in G are u_1 and u_2 , u_1 and u_3 , u_2 and u_4 , and u_3 and u_4 , and each of the pairs $f(u_1) = v_1$ and $f(u_2) = v_4$, $f(u_1) = v_1$ and $f(u_3) = v_3$, $f(u_2) = v_4$ and $f(u_4) = v_2$, and $f(u_3) = v_3$ and $f(u_4) = v_2$ consists of two adjacent vertices in H .

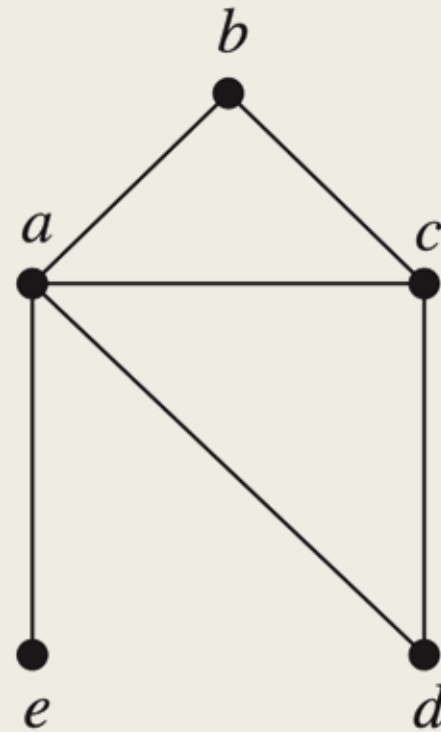


Isomorphism of Graphs: Example

Example. Show that the graphs G and H are not isomorphic.



G

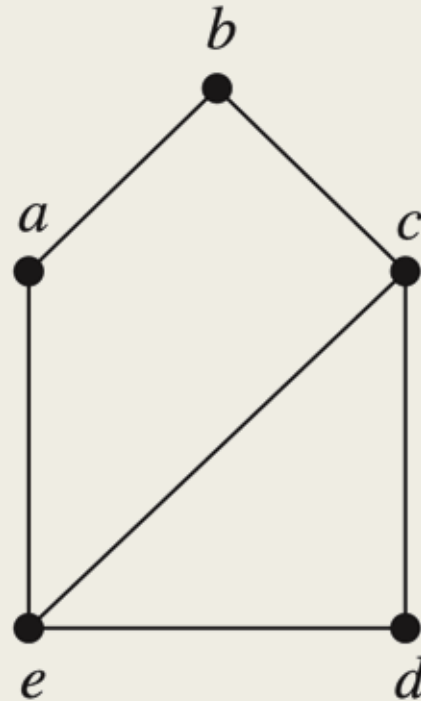


H

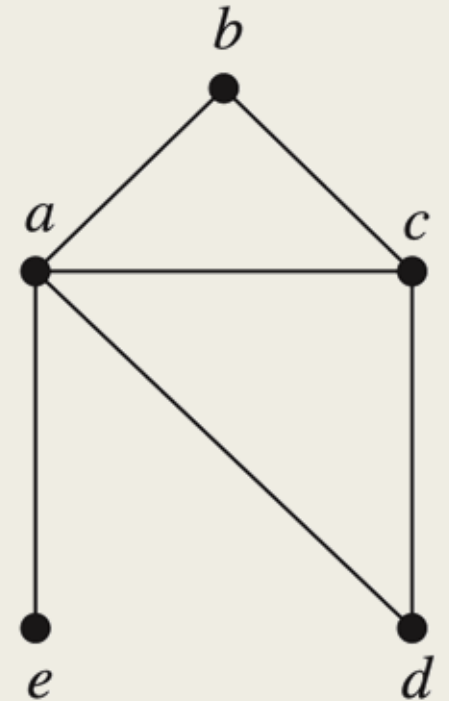
Isomorphism of Graphs: Example

Example. Show that the graphs G and H are not isomorphic.

Solution: Both G and H have 5 vertices and 6 edges. However, H has a vertex of degree one, namely, e , whereas G has no vertices of degree one. It follows that G and H are not isomorphic.



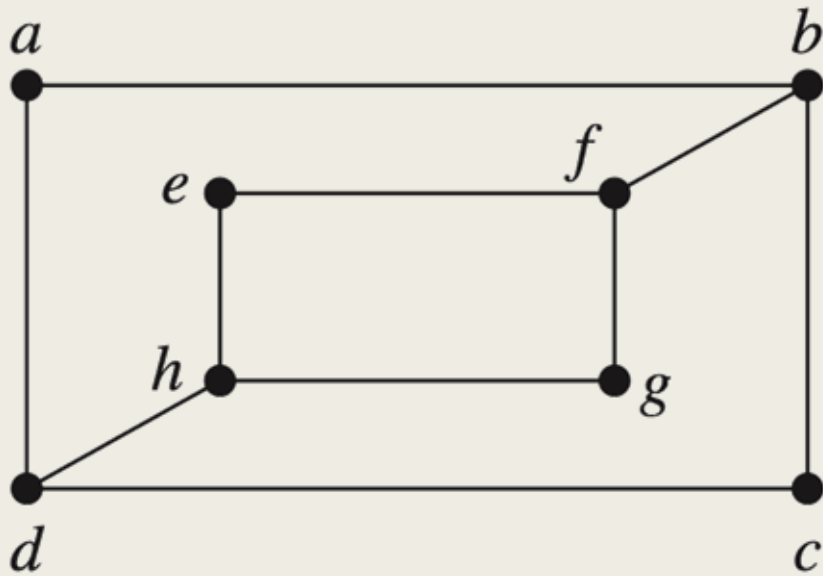
G



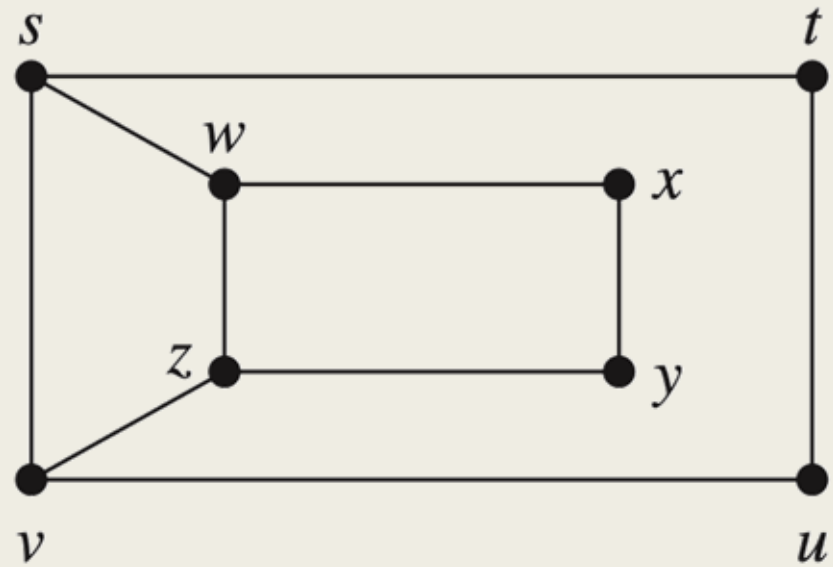
H

Isomorphism of Graphs: Example

Example. Determine whether the graphs G and H are isomorphic.



G



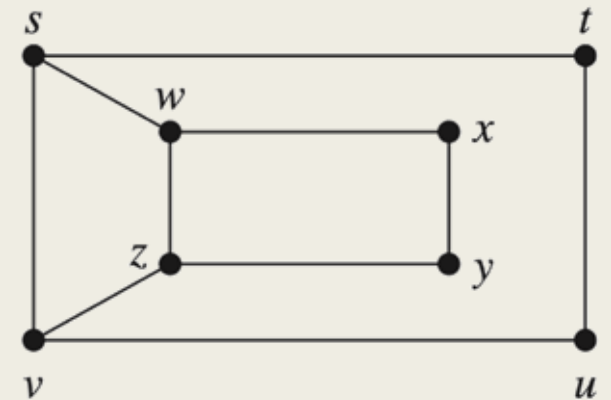
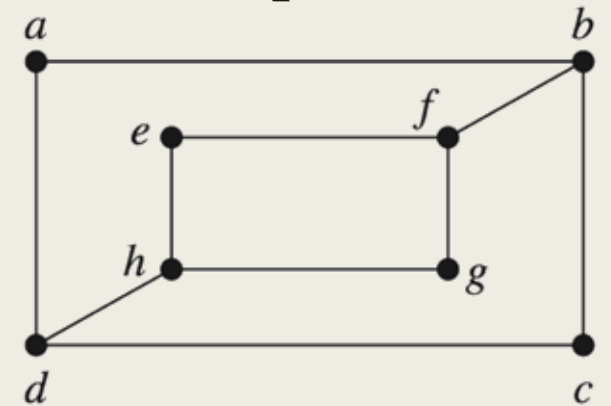
H

Isomorphism of Graphs: Example

Example. Determine whether the graphs G and H are isomorphic.

Solution: The graphs G and H both have 8 vertices and 10 edges. They also both have 4 vertices of degree 2 and 4 of degree 3.

However, G and H are not isomorphic. To see this, note that because $\deg(a) = 2$ in G , a must correspond to either t, u, x , or y in H , because these are the vertices of degree 2 in H . However, each of these 4 vertices in H is adjacent to another vertex of degree 2 in H , which is not true for a in G .



CONNECTIVITY

Paths, Connectedness in Undirected Graphs

Many problems can be modeled with paths formed by traveling along the edges of graphs.

For instance, the problem of determining whether a message can be sent between two computers using intermediate links can be studied with a graph model.

Problems of efficiently planning routes for mail delivery, garbage pickup, diagnostics in computer networks, and so on can be solved using models that involve paths in graphs.

Informally, a **path** is a sequence of edges that begins at a vertex of a graph and travels from vertex to vertex along edges of the graph. As the path travels along its edges, it visits the vertices along this path, that is, the endpoints of these edges.

DEFINITION 1. Let n be a nonnegative integer and G an undirected graph. A ***path of length n*** from u to v in G is a sequence of n edges $e_1, \dots, e_n$ of G for which there exists a sequence $x_0 = u, x_1, \dots, x_{n-1}, x_n = v$ of vertices such that e_i has, for $i = 1, \dots, n$, the endpoints x_{i-1} and x_i . When the graph is simple, we denote this path by its vertex sequence $x_0, x_1, \dots, x_n$ (because listing these vertices uniquely determines the path).

- The path is a ***circuit*** if it begins and ends at the same vertex, that is, if $u = v$, and has length greater than zero.
- The path or circuit is said to ***pass through*** the vertices $x_1, x_2, \dots, x_{n-1}$ or ***traverse*** the edges $e_1, e_2, \dots, e_n$.
- A path or circuit is ***simple*** if it does not contain the same edge more than once.

Paths

When it is not necessary to distinguish between multiple edges, we will denote a path $e_1, e_2, \dots, e_n$, where e_i is associated with $\{x_{i-1}, x_i\}$ for $i = 1, 2, \dots, n$ by its vertex sequence $x_0, x_1, \dots, x_n$. This notation identifies a path only as far as which vertices it passes through.

Consequently, it does not specify a unique path when there is more than one path that passes through this sequence of vertices, which will happen if and only if there are multiple edges between some successive vertices in the list.

NOTE that a path of length zero consists of a single vertex.

Remark. There is considerable variation of terminology concerning the concepts defined in Definition 1.

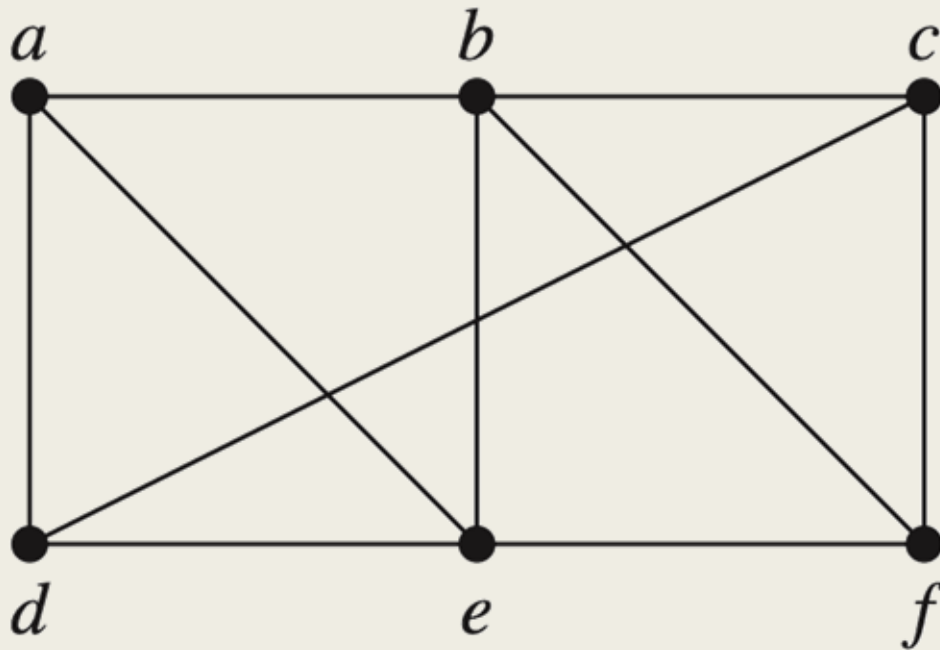
For instance, in some books, the term

- **walk** is used instead of *path*, where a walk is defined to be an alternating sequence of vertices and edges of a graph, $v_0, e_1, v_1, e_2, \dots, v_{n-1}, e_n, v_n$, where v_{i-1} and v_i are the endpoints of e_i for $i = 1, 2, \dots, n$.
- **closed walk** is used instead of *circuit* to indicate a walk that begins and ends at the same vertex, and
- **trail** is used to denote a walk that has no repeated edge (replacing the term *simple path*).
- **path** is often used for a trail with no repeated vertices, conflicting with the terminology in Definition 1.

Because of this variation in terminology, you will need to make sure which set of definitions are used in a particular book or article when you read about traversing edges of a graph.

Paths: Example

EXAMPLE. In the following simple graph a, d, c, f, e is a simple path of length 4, because $\{a, d\}, \{d, c\}, \{c, f\}$, and $\{f, e\}$ are all edges. However, d, e, c, a is not a path, because $\{e, c\}$ is not an edge.



Note that b, c, f, e, b is a circuit of length 4 because $\{b, c\}, \{c, f\}, \{f, e\}$, and $\{e, b\}$ are edges, and this path begins and ends at b . The path a, b, e, d, a, b , which is of length 5, is not simple because it contains the edge $\{a, b\}$ twice.

Paths

DEFINITION 2. Let n be a nonnegative integer and G a directed graph. A ***path*** of length n from u to v in G is a sequence of edges $e_1, e_2, \dots, e_n$ of G such that e_1 is associated with (x_0, x_1) , e_2 is associated with (x_1, x_2) , and so on, with e_n associated with (x_{n-1}, x_n) , where $x_0 = u$ and $x_n = v$. When there are no multiple edges in the directed graph, this path is denoted by its vertex sequence $x_0, x_1, x_2, \dots, x_n$.

- A path of length greater than zero that begins and ends at the same vertex is called a ***circuit*** or ***cycle***.
- A path or circuit is called ***simple*** if it does not contain the same edge more than once.

Example: Paths in Collaboration Graphs

In a collaboration graph, two people a and b are connected by a path when there is a sequence of people starting with a and ending with b such that the endpoints of each edge in the path are people who have collaborated.

In the academic collaboration graph of people who have written papers in mathematics, the **Erdo's number** of a person m is the length of the shortest path between m and the extremely prolific (>1400 papers) mathematician Paul Erdo's.

That is, the Erdo's number of a mathematician is the length of the shortest chain of mathematicians that begins with Paul Erdo's and ends with this mathematician, where each adjacent pair of mathematicians have written a joint paper. The number of mathematicians with each Erdo's number as of early 2006, according to the Erdo's Number Project, is shown in Table 1.

TABLE 1 The Number of Mathematicians with a Given Erdős Number (as of early 2006).

<i>Erdős Number</i>	<i>Number of People</i>
0	1
1	504
2	6,593
3	33,605
4	83,642
5	87,760
6	40,014
7	11,591
8	3,146
9	819
10	244
11	68
12	23
13	5

Connectedness in Undirected Graphs

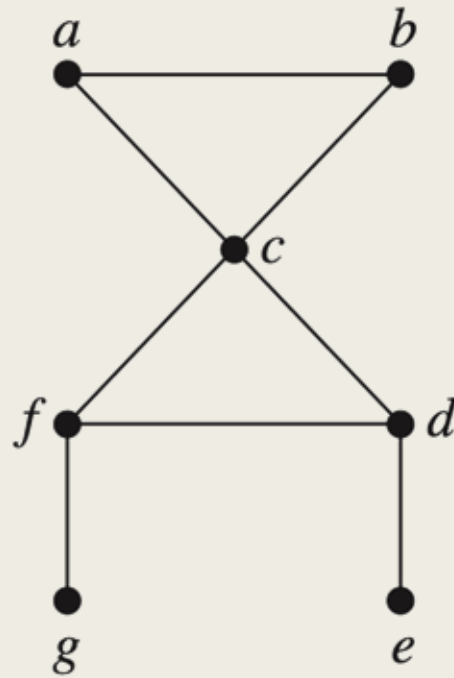
Q. When is there always a path between two vertices in the graph?

DEFINITION. An undirected graph is called ***connected*** if there is a path between every pair of distinct vertices of the graph.

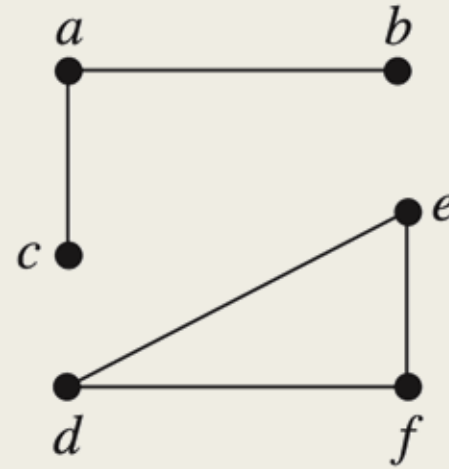
- An undirected graph that is not connected is called ***disconnected***.
- We say that we ***disconnect a graph*** when we remove vertices or edges, or both, to produce a disconnected subgraph.

Connectedness in Undirected Graphs

EXAMPLE. The graph G_1 is connected. However, the graph G_2 is disconnected.



G_1



G_2

Connectedness in Undirected Graphs

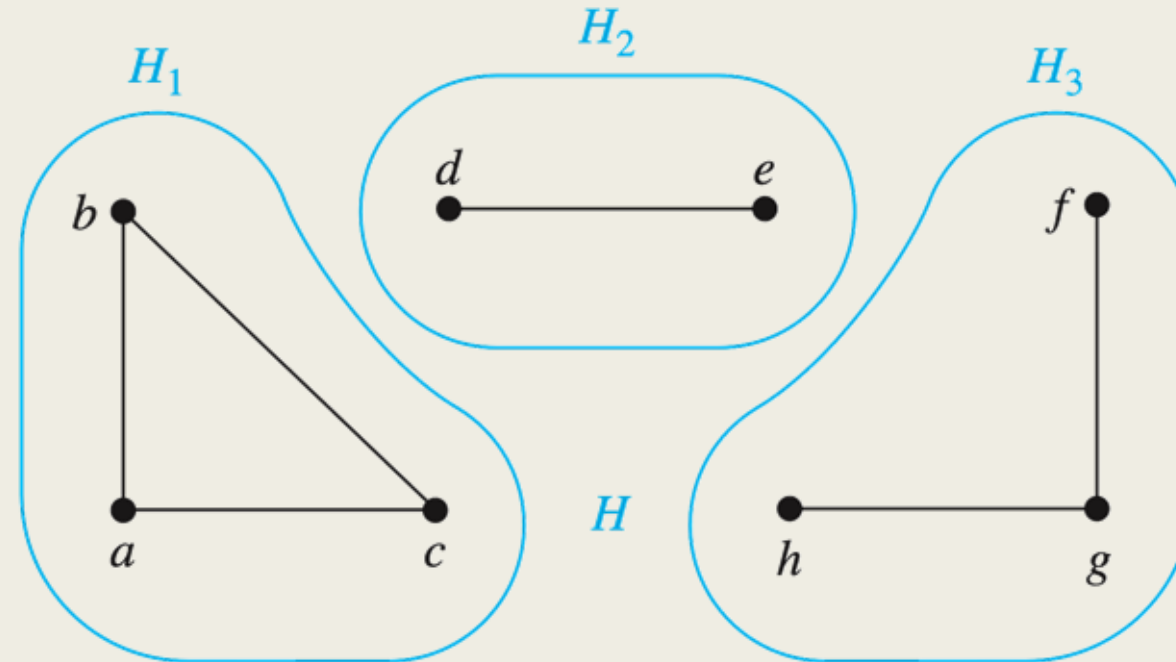
THEOREM 1. There is a simple path between every pair of distinct vertices of a connected undirected graph.

A *connected component* of a graph G is a connected subgraph of G that is not a proper subgraph of another connected subgraph of G . That is, a connected component of a graph G is a maximal connected subgraph of G . A graph G that is not connected has two or more connected components that are disjoint and have G as their union.

Connected Components: Example

EXAMPLE. What are the connected components of the graph H ?

Solution: The graph H is the union of three disjoint connected subgraphs H_1 , H_2 , and H_3 , which are the connected components of H .



How Connected is a Graph?

Suppose that a graph represents a computer network. Knowing that this graph is connected tells us that any two computers on the network can communicate.

However, we would also like to understand how reliable this network is.

For instance, will it still be possible for all computers to communicate after a router or a communications link fails?

To answer this and similar questions, we now develop some new concepts.

How Connected is a Graph?

Sometimes the removal from a graph of a vertex and all incident edges produces a subgraph with more connected components. Such vertices are called **cut vertices** (or **articulation points**). The removal of a cut vertex from a connected graph produces a subgraph that is not connected.

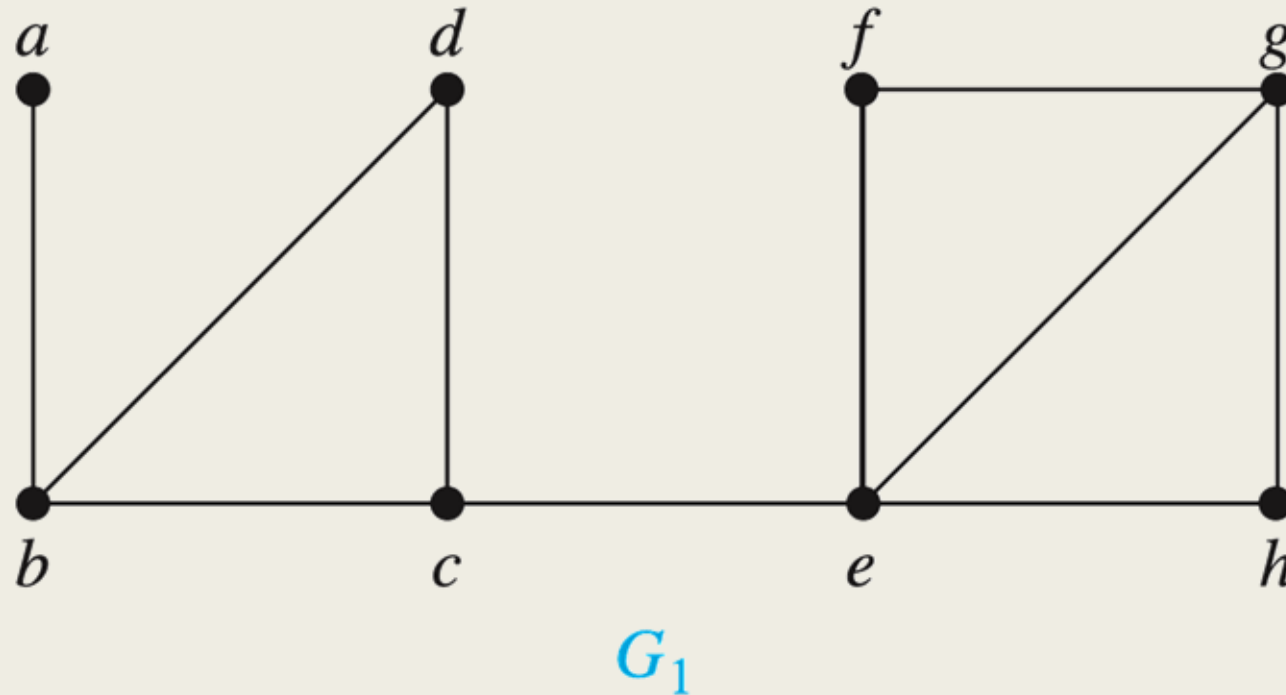
Analogously, an edge whose removal produces a graph with more connected components than in the original graph is called a **cut edge** or **bridge**.

NOTE that in a graph representing a computer network, a cut vertex and a cut edge represent an essential router and an essential link that cannot fail for all computers to be able to communicate.

Connected Graph: Example

EXAMPLE. Find the cut vertices and cut edges in the graph G_1 .

Solution: The cut vertices of G_1 are b, c , and e . The cut edges are $\{a, b\}$ and $\{c, e\}$.



Vertex Connectivity

Not all graphs have cut vertices.

For example, the complete graph K_n , where $n \geq 3$, has no cut vertices. When you remove a vertex from K_n and all edges incident to it, the resulting subgraph is the complete graph K_{n-1} , a connected graph.

- Connected graphs without cut vertices are called *nonseparable graphs*, and can be thought of as more connected than those with a cut vertex.
- A subset V' of the vertex set V of $G = (V, E)$ is a *vertex cut*, or *separating set*, if $G - V'$ is disconnected.

NOTE that every connected graph, except a complete graph, has a vertex cut

Vertex Connectivity

- We define the *vertex connectivity* of a noncomplete graph G , denoted by $\kappa(G)$, as the minimum number of vertices in a vertex cut.

When G is a complete graph, it has no vertex cuts, because removing any subset of its vertices and all incident edges still leaves a complete graph. Consequently, we cannot define $\kappa(G)$ as the minimum number of vertices in a vertex cut when G is complete. Instead, we set $\kappa(K_n) = n - 1$, the number of vertices needed to be removed to produce a graph with a single vertex.

Vertex Connectivity

Consequently, for every graph G , $\kappa(G)$ is minimum number of vertices that can be removed from G to either disconnect G or produce a graph with a single vertex.

We have

- $0 \leq \kappa(G) \leq n - 1$ if G has n vertices,
- $\kappa(G) = 0$ if and only if G is disconnected or $G = K_1$, and
- $\kappa(G) = n - 1$ if and only if G is complete.

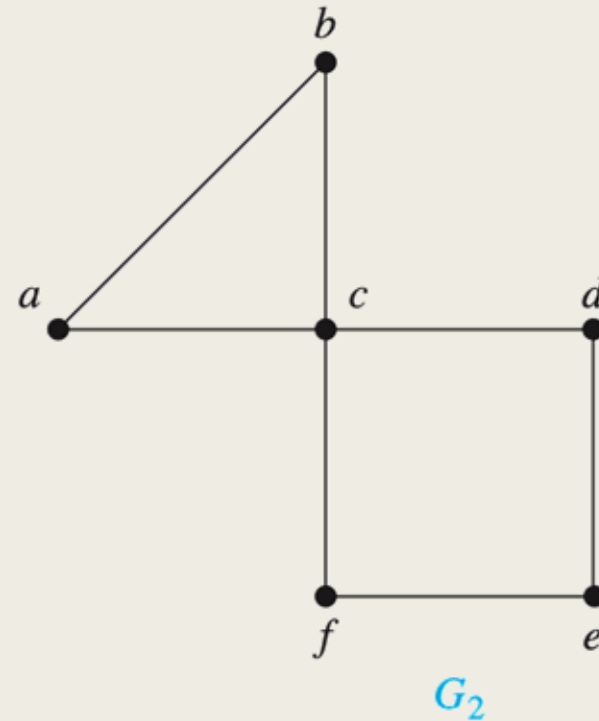
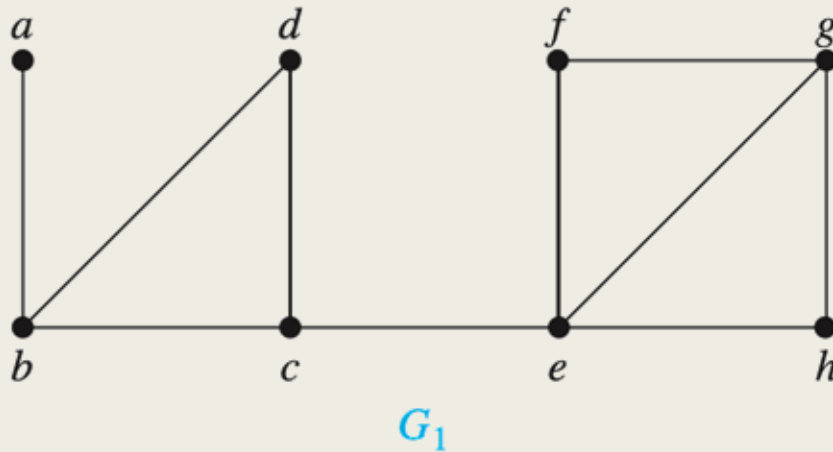
The larger $\kappa(G)$ is, the more connected we consider G to be.

- Disconnected graphs and K_1 have $\kappa(G) = 0$,
- connected graphs with cut vertices and K_2 have $\kappa(G) = 1$,
- graphs without cut vertices that can be disconnected by removing two vertices and K_3 have $\kappa(G) = 2$, and so on.

Vertex Connectivity: Example

EXAMPLE. Find the vertex connectivity for each of the graphs.

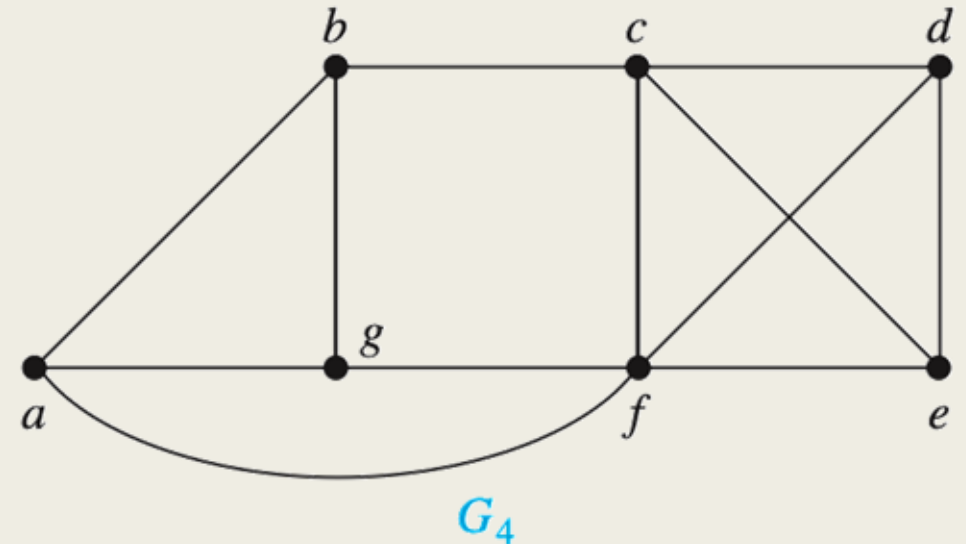
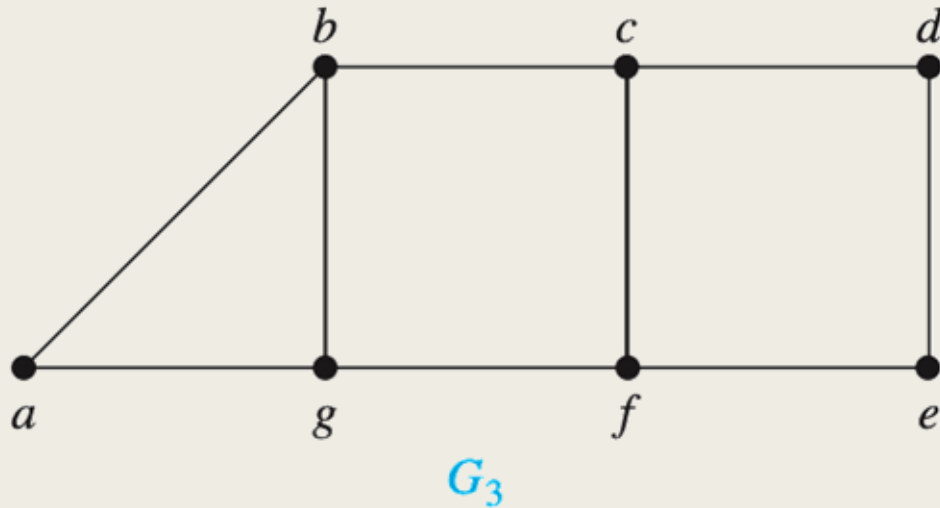
Answer: Each of these graphs has positive vertex connectivity. Because G_1 is a connected graph with a cut vertex, as shown in previous Example, we know that $\kappa(G_1) = 1$. Similarly, $\kappa(G_2) = 1$, because c is a cut vertex of G_2 .



Vertex Connectivity: Example

EXAMPLE. Find the vertex connectivity for each of the graphs.

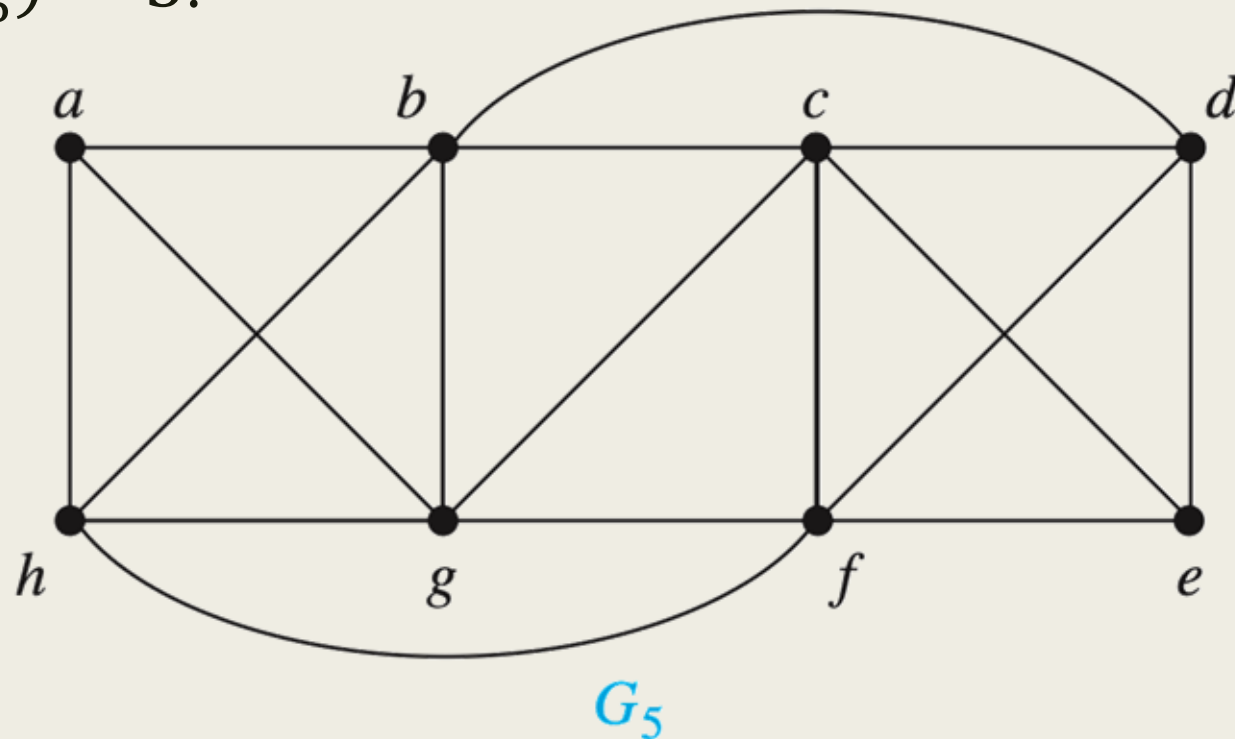
Answer: G_3 has no cut vertices, but that $\{b, g\}$ is a vertex cut. Hence, $\kappa(G_3) = 2$. Similarly, because G_4 has a vertex cut of size two, $\{c, f\}$, but no cut vertices. It follows that $\kappa(G_4) = 2$.



Vertex Connectivity: Example

EXAMPLE. Find the vertex connectivity for the following graph.

Answer: G_5 has no vertex cut of size two, but $\{b, c, f\}$ is a vertex cut of G_5 . Hence, $\kappa(G_5) = 3$.



Edge Connectivity

We can also measure the connectivity of a connected graph $G = (V, E)$ in terms of the minimum number of edges that we can remove to disconnect it.

- If a graph has a cut edge, then we need only remove it to disconnect G .
- If G does not have a cut edge, we look for the smallest set of edges that can be removed to disconnect it.
- A set of edges E' is called an ***edge cut*** of G if the subgraph $G - E'$ is disconnected.
- The ***edge connectivity*** of a graph G , denoted by $\lambda(G)$, is the minimum number of edges in an edge cut of G .

Edge Connectivity

This defines $\lambda(G)$ for all connected graphs with more than one vertex because it is always possible to disconnect such a graph by removing all edges incident to one of its vertices.

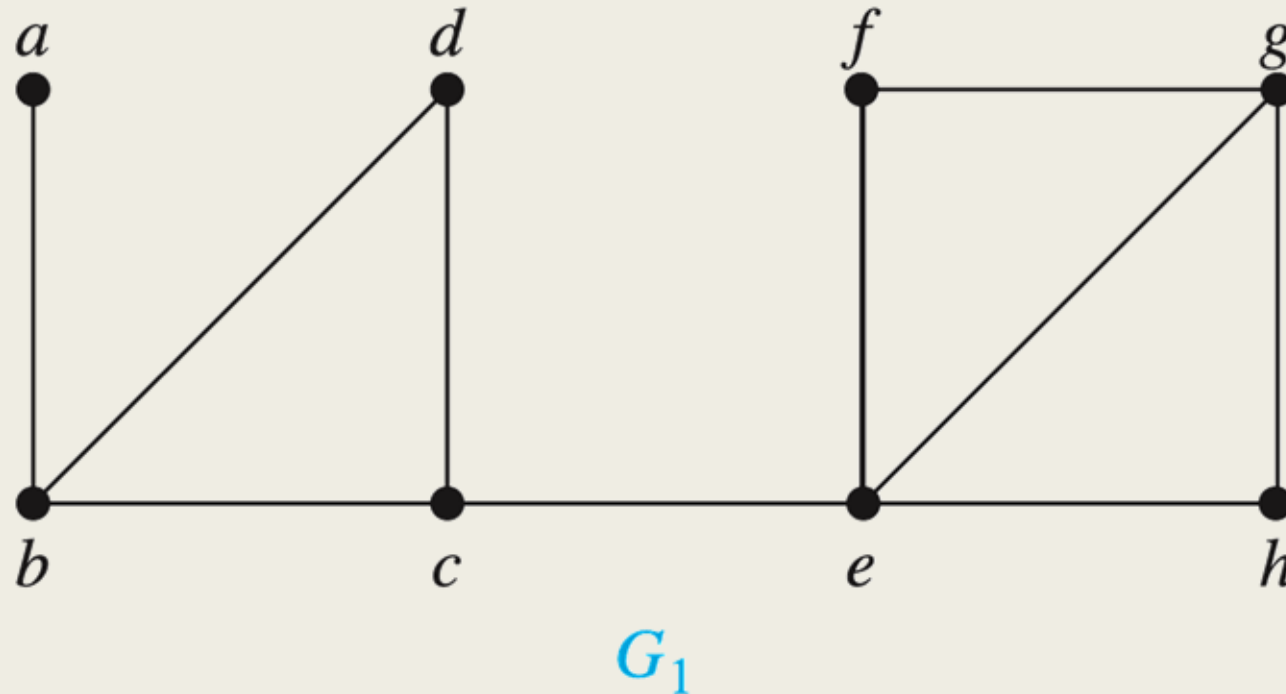
NOTE that

- $\lambda(G) = 0$ if G is not connected.
- $\lambda(G) = 0$ if G is a graph consisting of a single vertex.
- if G is a graph with n vertices, then $0 \leq \lambda(G) \leq n - 1$.
- $\lambda(G) = n - 1$ where G is a graph with n vertices if and only if $G = K_n$.

Edge Connectivity: Example

EXAMPLE. Find the edge connectivity of the following graph.

Answer: G_1 has a cut edge, so $\lambda(G_1) = 1$.

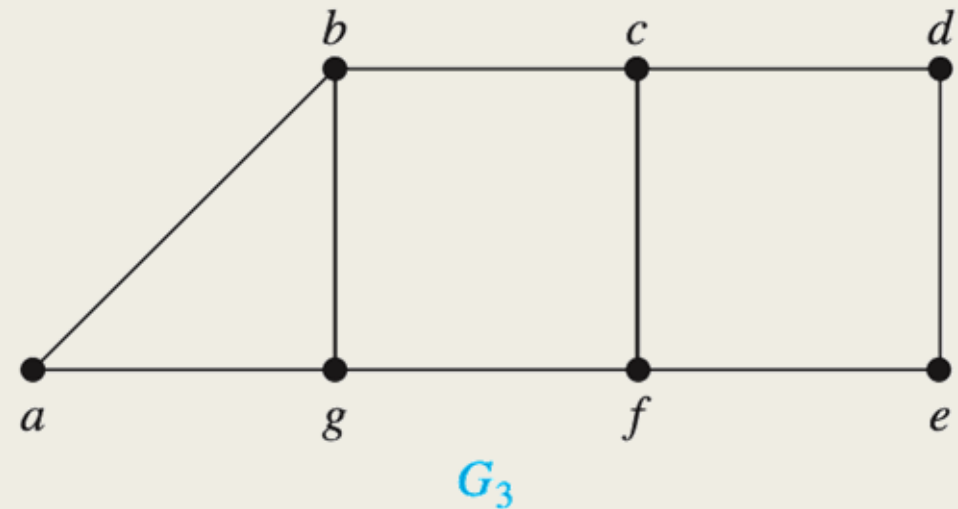
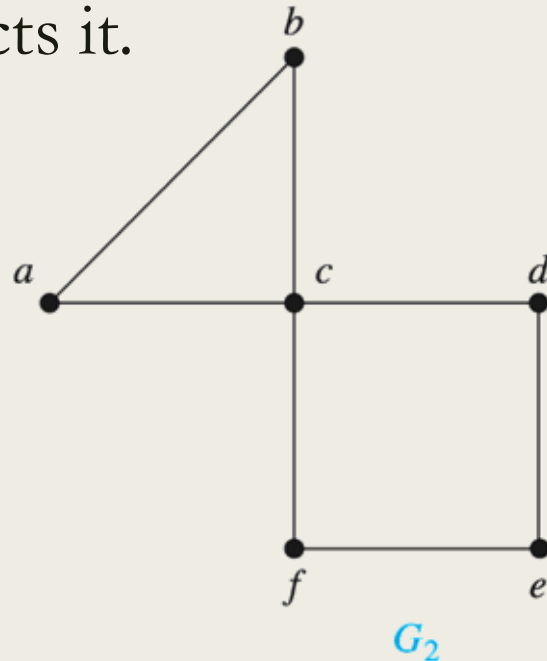


Edge Connectivity: Example

EXAMPLE. Find the edge connectivity of the following graphs.

Answer: The graph G_2 has no cut edges, but the removal of the two edges $\{a, b\}$ and $\{a, c\}$ disconnects it. Hence, $\lambda(G_2) = 2$.

Similarly, $\lambda(G_3) = 2$, the removal of the two edges $\{b, c\}$ and $\{f, g\}$ disconnects it.

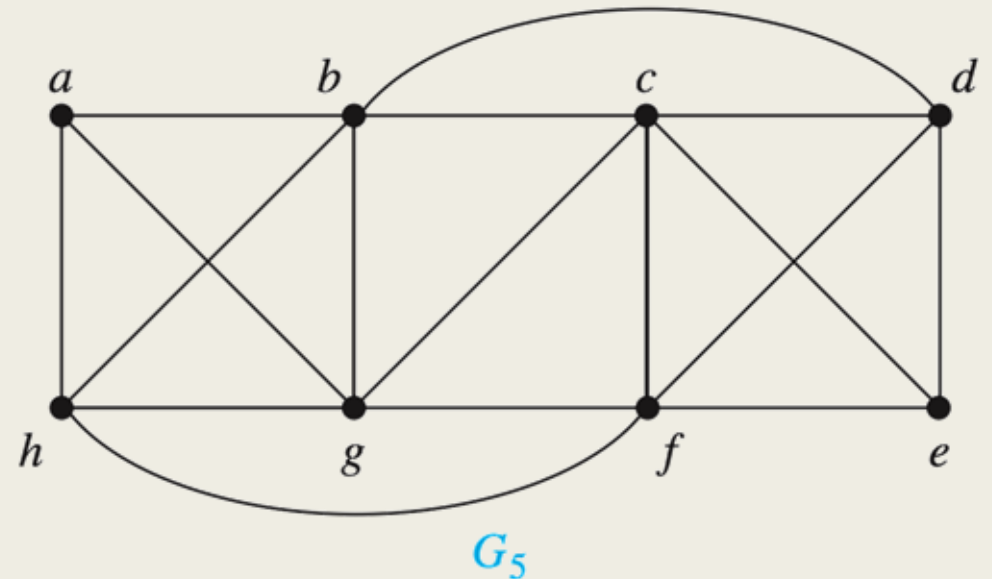
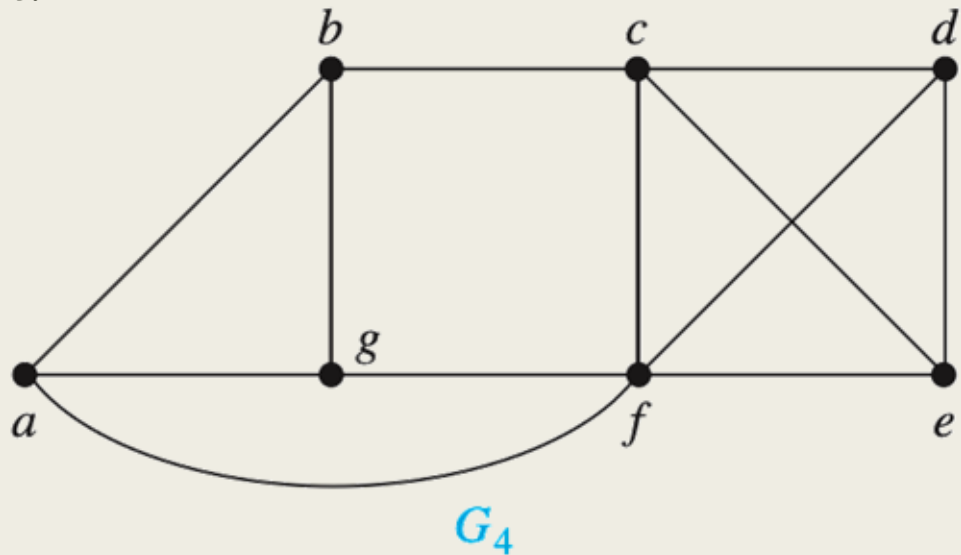


Edge Connectivity: Example

EXAMPLE. Find the edge connectivity of the following graphs.

Answer: The removal of the three edges $\{b, c\}$, $\{a, f\}$, and $\{f, g\}$ disconnects G_4 . Hence, $\lambda(G_4) = 3$.

$\lambda(G_5) = 3$, because the removal of $\{a, b\}$, $\{a, g\}$, and $\{a, h\}$ disconnect it.



An Inequality for Vertex connectivity and Edge connectivity

When $G = (V, E)$ is a noncomplete connected graph with at least 3 vertices, the minimum degree of a vertex of G is an upper bound for both the vertex connectivity of G and the edge connectivity of G . That is,

$$\kappa(G) \leq \min_{v \in V} \deg(v) \quad \text{and} \quad \lambda(G) \leq \min_{v \in V} \deg(v)$$

To see this, observe that deleting all the neighbors of a fixed vertex of minimum degree disconnects G , and deleting all the edges that have a fixed vertex of minimum degree as an endpoint disconnects G .

An Inequality for Vertex connectivity and Edge connectivity

- $\kappa(G) \leq \lambda(G)$ when G is a connected noncomplete graph.
- $\kappa(K_n) = \lambda(K_n) = \min_{v \in V} \{\deg(v)\} = n - 1$ when n is a positive integer
- $\kappa(G) = \lambda(G) = 0$ when G is a disconnected graph.

Putting these facts together, establishes that for all graphs G ,

$$\kappa(G) \leq \lambda(G) \leq \min_{v \in V} \deg(v)$$

Connectedness in Directed Graphs

DEFINITION. A directed graph is *strongly connected* if there is a path from a to b and from b to a whenever a and b are vertices in the graph.

For a directed graph to be strongly connected there must be a sequence of directed edges from any vertex in the graph to any other vertex. A directed graph can fail to be strongly connected but still be in “one piece.”

DEFINITION. A directed graph is *weakly connected* if there is a path between every two vertices in the underlying undirected graph.

That is, a directed graph is weakly connected if and only if there is always a path between two vertices when the directions of the edges are disregarded.

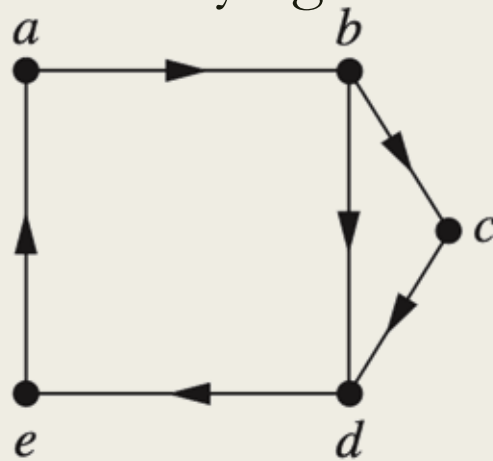
Clearly, any strongly connected directed graph is also weakly connected.

Connectedness in Directed Graphs: Example

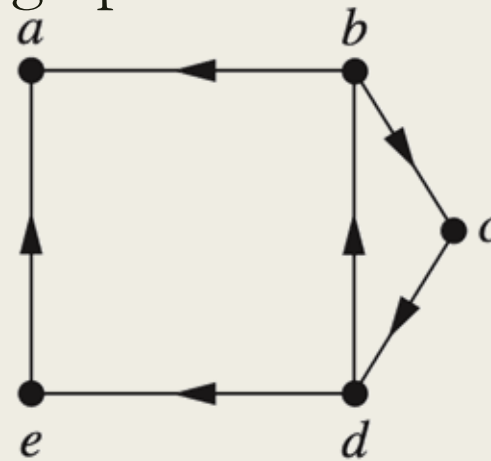
EXAMPLE. Are the directed graphs G and H strongly connected? Are they weakly connected?

Answer: G is strongly connected because there is a path between any two vertices in this directed graph. Hence, G is also weakly connected.

The graph H is not strongly connected. There is no directed path from a to b in this graph. However, H is weakly connected, because there is a path between any two vertices in the underlying undirected graph of H .



G



H

Paths and Isomorphism

There are several ways that paths and circuits can help determine whether two graphs are isomorphic.

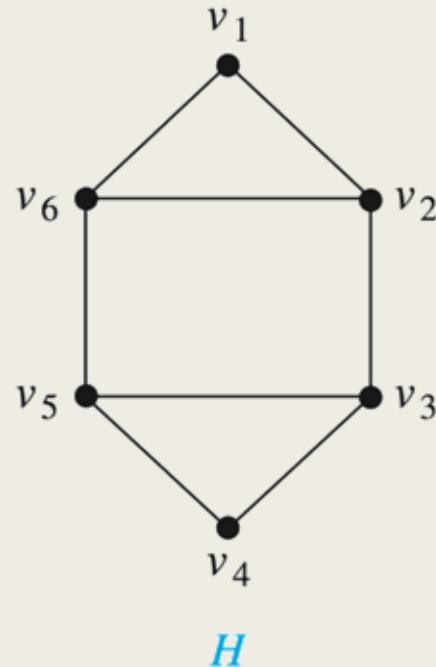
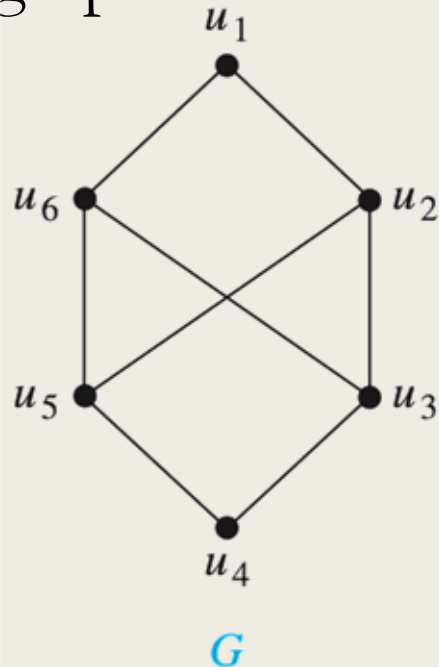
For example, paths can be used to construct mappings that may be isomorphisms.

A useful isomorphic invariant for simple graphs is the existence of a simple circuit of length k , where k is a positive integer greater than 2.

Paths and Isomorphism: Example

EXAMPLE. Determine whether the graphs G and H are isomorphic.

Answer: Both G and H have 6 vertices and 8 edges. Each has 4 vertices of degree 3, and 2 vertices of degree 2. So, the three invariants – number of vertices, number of edges, and degrees of vertices – all agree for the two graphs.



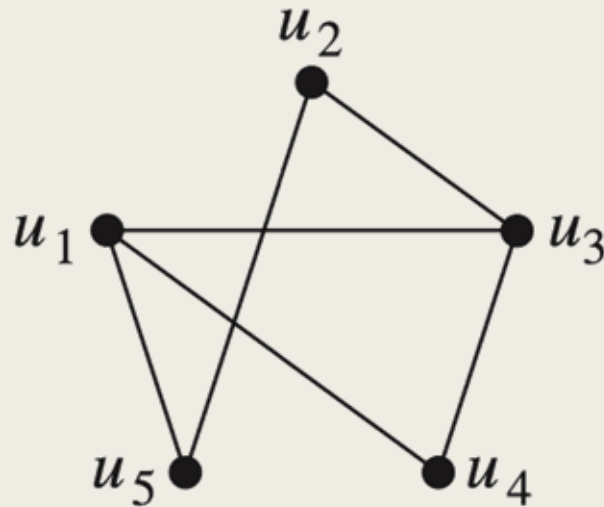
However, H has a simple circuit of length 3, namely, v_1, v_2, v_6, v_1 , whereas G has no simple circuit of length 3, as can be determined by inspection.

Because the existence of a simple circuit of length 3 is an isomorphic invariant, G and H are not isomorphic.

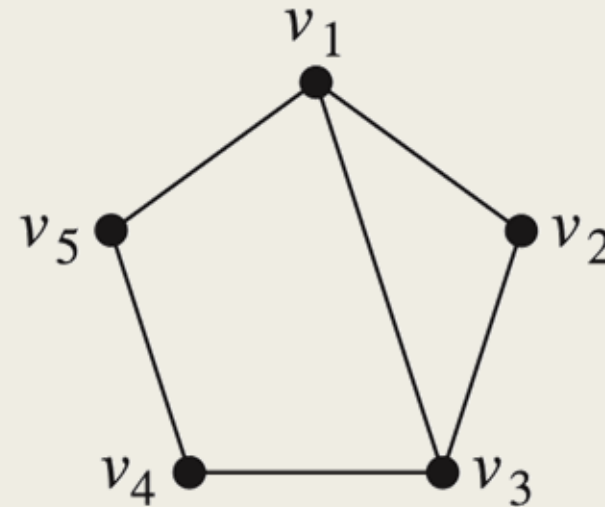
Paths and Isomorphism: Example

EXAMPLE. Determine whether the graphs G and H are isomorphic.

Answer: Both G and H have 5 vertices and 6 edges, both have 2 vertices of degree 3 and 3 vertices of degree 2, and both have a simple circuit of length 3, a simple circuit of length 4, and a simple circuit of length 5. Because all these isomorphic invariants agree, G and H may be isomorphic.



G

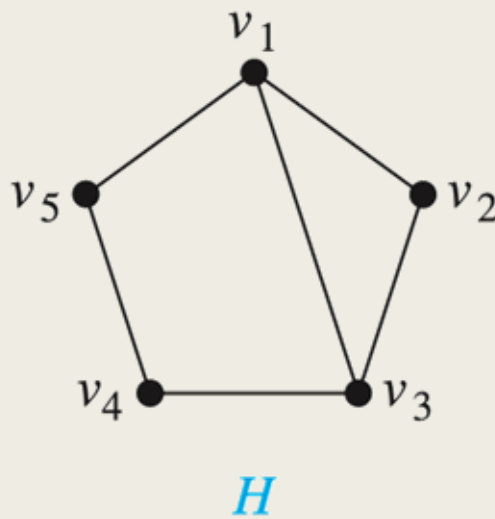
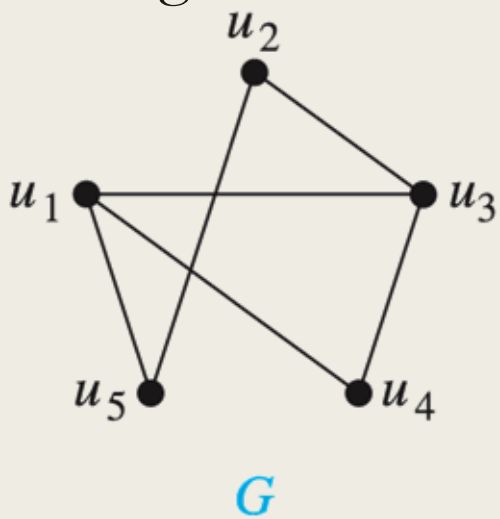


H

Paths and Isomorphism: Example

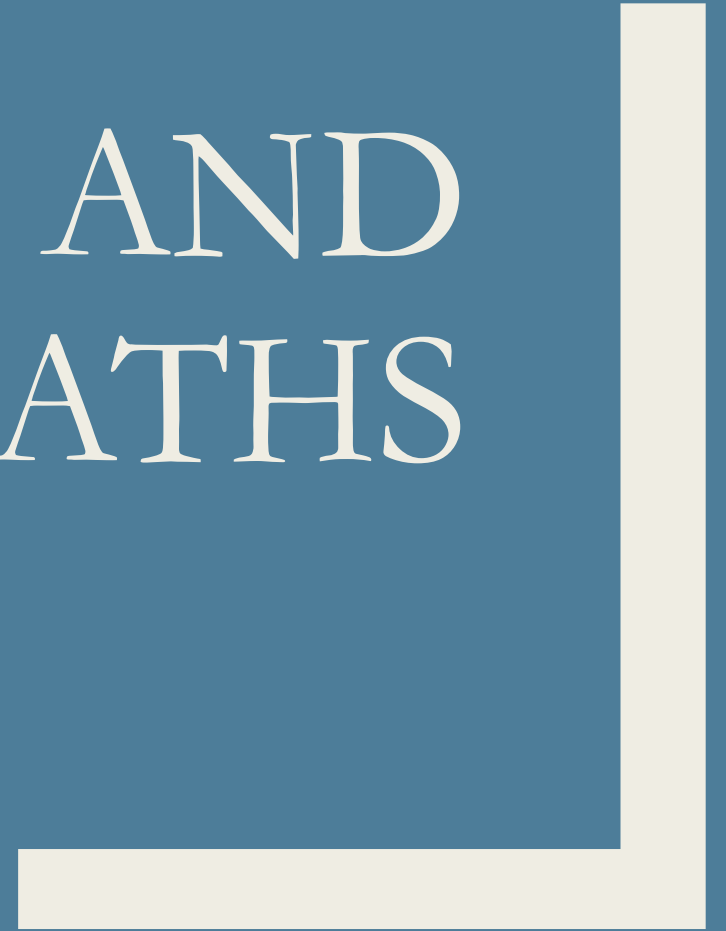
EXAMPLE (cont.). To find a possible isomorphism, we can follow paths that go through all vertices so that the corresponding vertices in the two graphs have the same degree.

For example, the paths u_1, u_4, u_3, u_2, u_5 in G and v_3, v_2, v_1, v_5, v_4 in H both go through every vertex in the graph; start at a vertex of degree three; go through vertices of degrees two, three, and two, respectively; and end at a vertex of degree two.



By following these paths through the graphs, we define the mapping f with $f(u_1) = v_3$, $f(u_4) = v_2$, $f(u_3) = v_1$, $f(u_2) = v_5$, and $f(u_5) = v_4$. Note that f is an isomorphism, so G and H are isomorphic.

EULER AND HAMILTON PATHS



Euler and Hamilton Paths

Q1. Can we travel along the edges of a graph starting at a vertex and returning to it by traversing each edge of the graph exactly once?

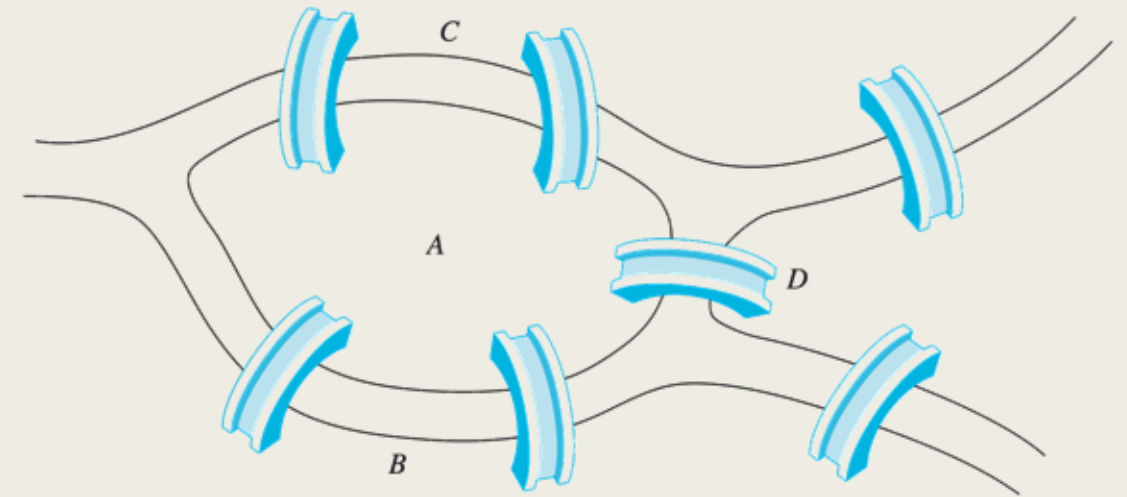
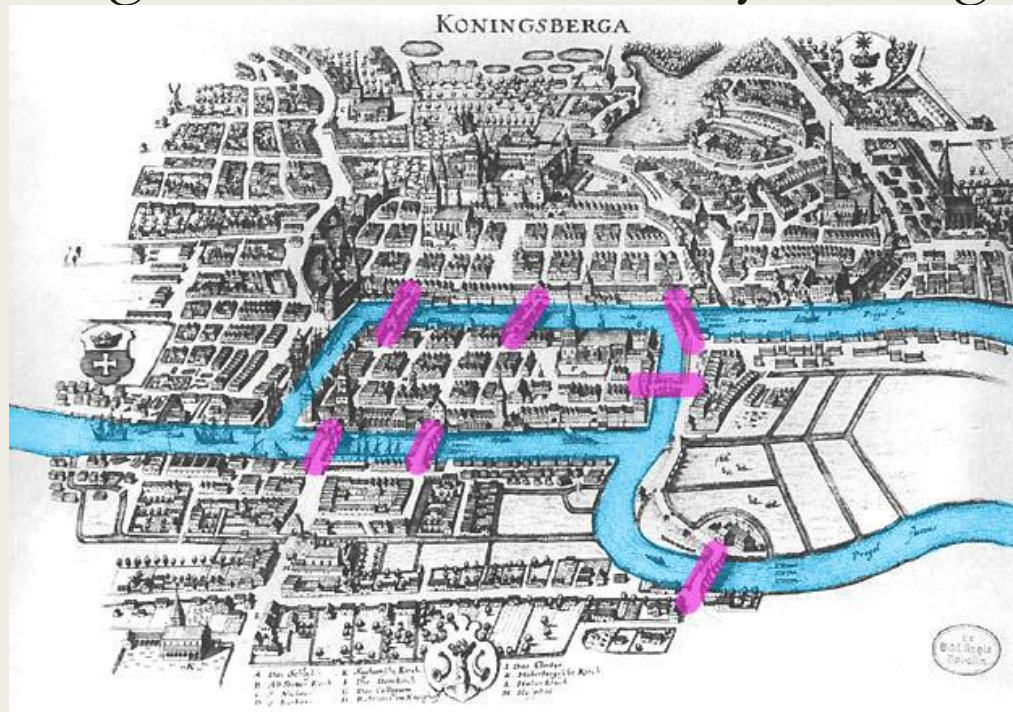
Q2. Similarly, can we travel along the edges of a graph starting at a vertex and returning to it while visiting each vertex of the graph exactly once?

Although these questions seem to be similar,

- the first question, which asks whether a graph has an ***Euler circuit***, can be easily answered simply by examining the degrees of the vertices of the graph, while the
- second question, which asks whether a graph has a ***Hamilton circuit***, is quite difficult to solve for most graphs.

Seven Bridges of Königsberg

The town of Königsberg, Prussia (now called Kaliningrad and part of the Russian Federation), was divided into 4 sections by the branches of the Pregel River. These 4 sections included the 2 regions on the banks of the Pregel, Kneiphof Island, and the region between the two branches of the Pregel. In the 18th century 7 bridges connected these regions.

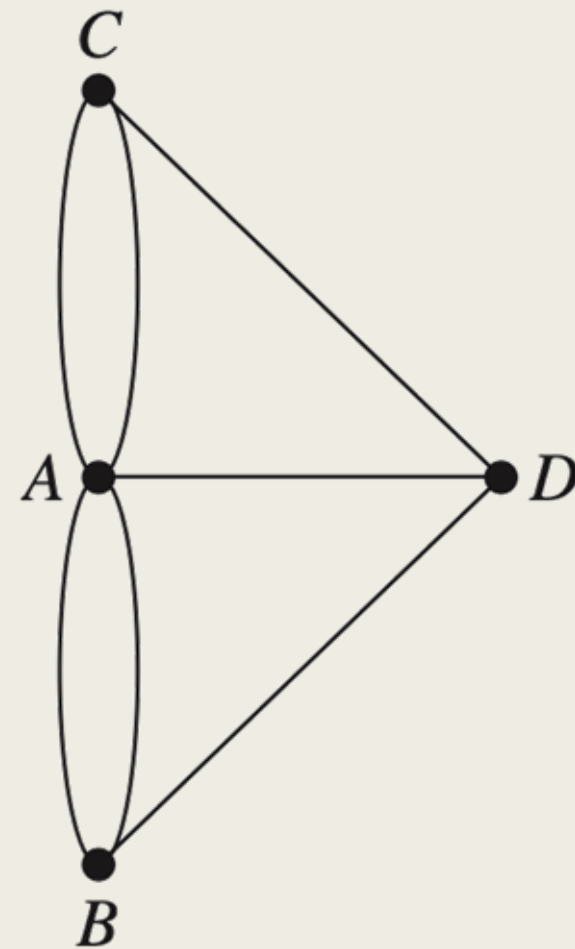


Seven Bridges of Königsberg

The townspeople took long walks through town on Sundays. They wondered whether it was possible to start at some location in the town, travel across all the bridges once without crossing any bridge twice and return to the starting point.

The Swiss mathematician Leonhard Euler solved this problem. His solution, published in 1736, may be the 1st use of graph theory.

Euler studied this problem using the multigraph obtained when four regions are represented by vertices and the bridges by edges.



Euler Paths and Circuits

The problem of traveling across every bridge without crossing any bridge more than once can be rephrased in terms of this model.

The question becomes:

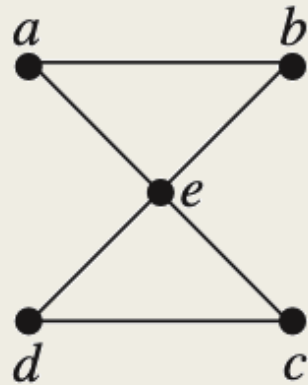
Q. Is there a simple circuit in this multigraph that contains every edge?

DEFINITION. An *Euler circuit* in a graph G is a simple circuit containing every edge of G . An *Euler path* in G is a simple path containing every edge of G .

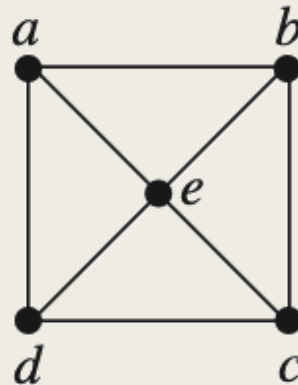
Euler Paths and Circuits: Example

EXAMPLE. Which of the following undirected graphs have an Euler circuit? Of those that do not, which have an Euler path?

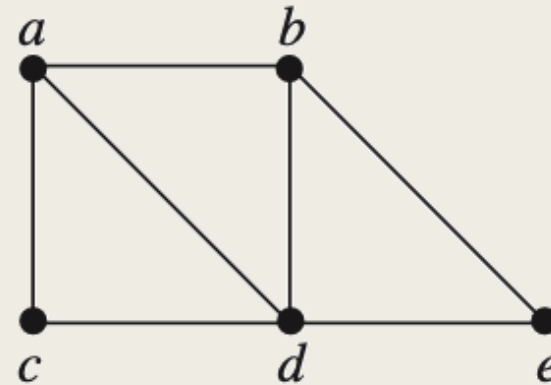
Answer: The graph G_1 has an Euler circuit, for example, a, e, c, d, e, b, a . Neither of the graphs G_2 or G_3 has an Euler circuit. However, G_3 has an Euler path, namely, a, c, d, e, b, d, a, b . G_2 does not have an Euler path.



G_1



G_2

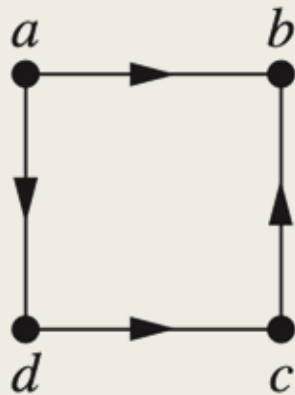


G_3

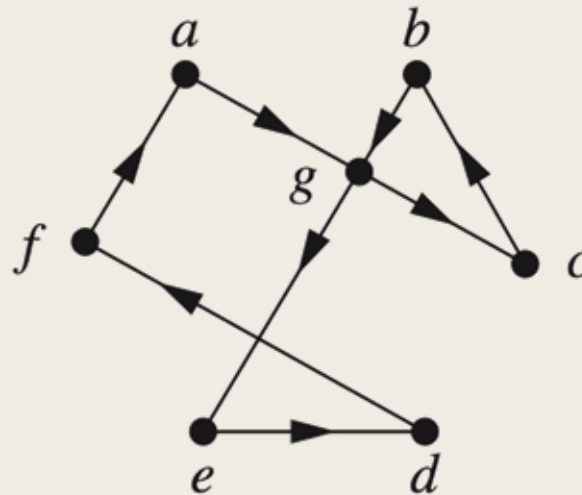
Euler Paths and Circuits: Example

EXAMPLE. Which of the following directed graphs have an Euler circuit? Of those that do not, which have an Euler path?

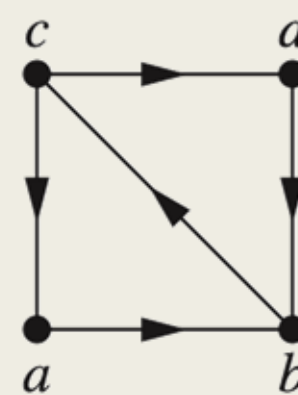
Answer: The graph H_2 has an Euler circuit, for example, $a, g, c, b, g, e, d, f, a$. Neither H_1 nor H_3 has an Euler circuit. H_3 has an Euler path, namely, c, a, b, c, d, b , but H_1 does not.



H_1



H_2



H_3

Necessary and sufficient conditions for Euler circuits and paths

There are simple criteria for determining whether a multigraph has an Euler circuit or an Euler path. Euler discovered them when he solved the famous Königsberg bridge problem.

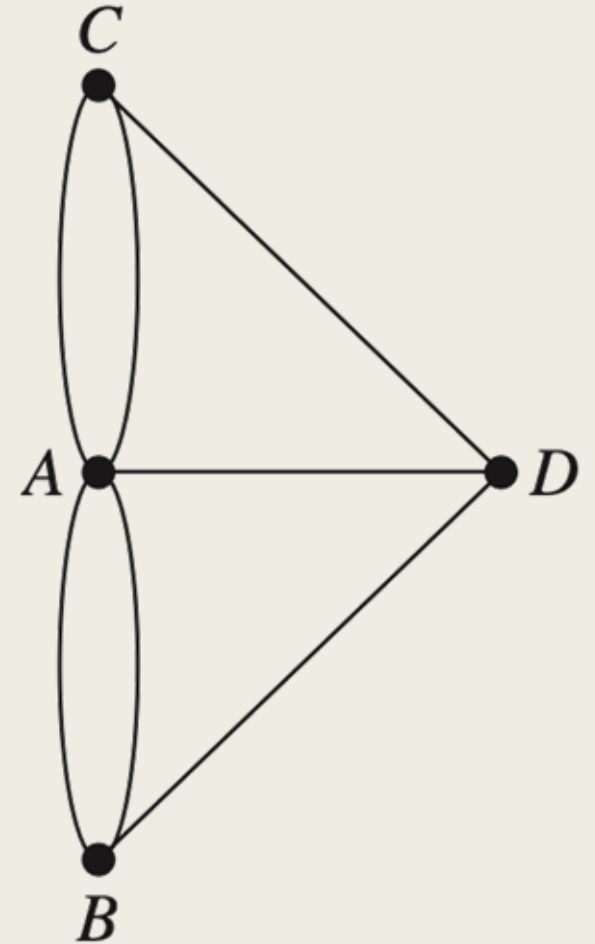
THEOREM 1. A connected multigraph with at least two vertices has an Euler circuit if and only if each of its vertices has even degree.

Königsberg bridge problem

We can now solve the Königsberg bridge problem.

Because the multigraph representing these bridges, has 4 vertices of odd degree, it does not have an Euler circuit by Theorem 1.

There is no way to start at a given point, cross each bridge exactly once, and return to the starting point.

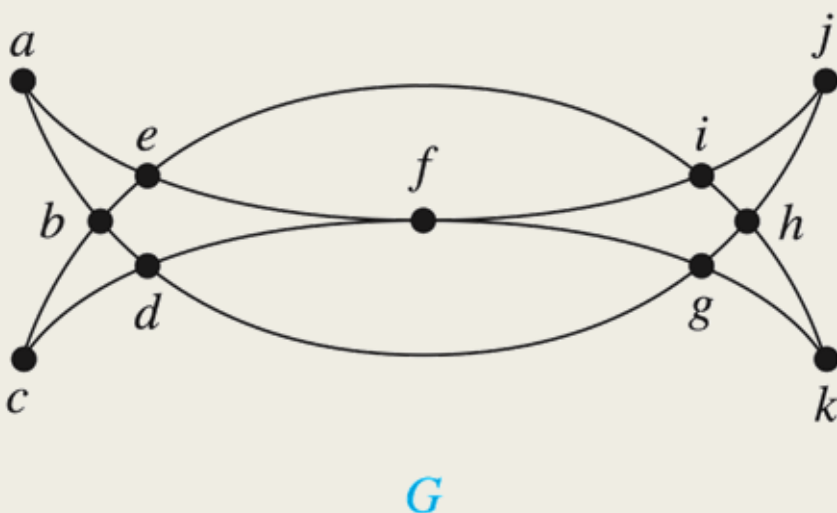


Euler circuits and paths: Example



Many puzzles ask you to draw a picture in a continuous motion without lifting a pencil so that no part of the picture is retraced. We can solve such puzzles using Euler circuits and paths. For example, can *Mohammed's scimitars*, be drawn in this way, where the drawing begins and ends at the same point?

We can solve this problem because the graph G has an Euler circuit. It has such a circuit because all its vertices have even degree. We will construct an Euler circuit. First, we form the circuit $a, b, d, c, b, e, i, f, e, a$. We obtain the subgraph H by deleting the edges in this circuit and all vertices that become isolated when these edges are removed.



Then we form the circuit $d, g, h, j, i, h, k, g, f, d$ in H . After forming this circuit we have used all edges in G . Splicing this new circuit into the first circuit at the appropriate place produces the Euler circuit $a, b, d, g, h, j, i, h, k, g, f, d, c, b, e, i, f, e, a$. This circuit gives a way to draw the scimitars without lifting the pencil or retracing part of the picture.

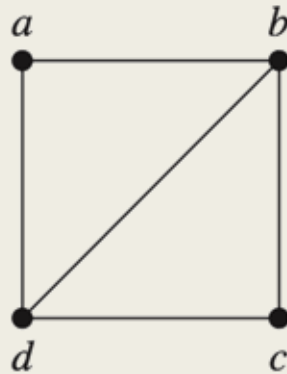
Euler circuits and paths

THEOREM. A connected multigraph has an Euler path but not an Euler circuit if and only if it has exactly two vertices of odd degree.

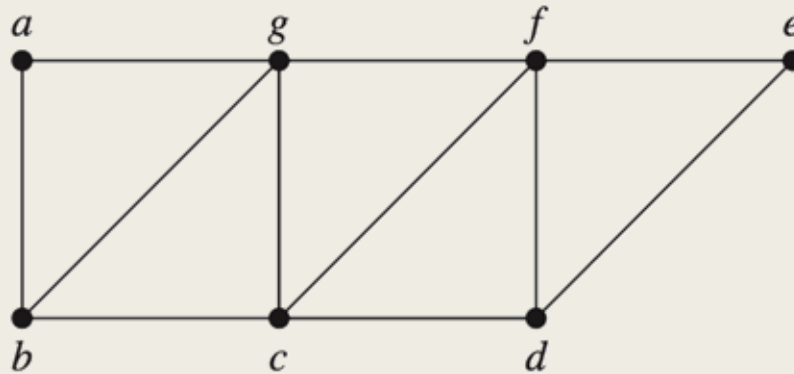
Euler circuits and paths: Example

EXAMPLE. Which of the following graphs have an Euler path?

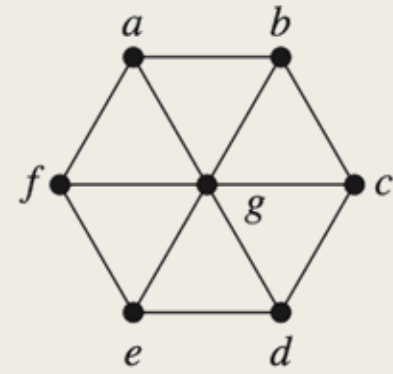
Solution: G_1 contains exactly 2 vertices of odd degree, namely, b and d . Hence, it has an Euler path that must have b and d as its endpoints. One such Euler path is d, a, b, c, d, b . Similarly, G_2 has exactly 2 vertices of odd degree, namely, b and d . So it has an Euler path that must have b and d as endpoints. One such Euler path is $b, a, g, f, e, d, c, g, b, c, f, d$. G_3 has no Euler path because it has 6 vertices of odd degree.



G_1



G_2



G_3

Hamilton Paths and Circuits

We have developed necessary and sufficient conditions for the existence of paths and circuits that contain every edge of a multigraph exactly once.

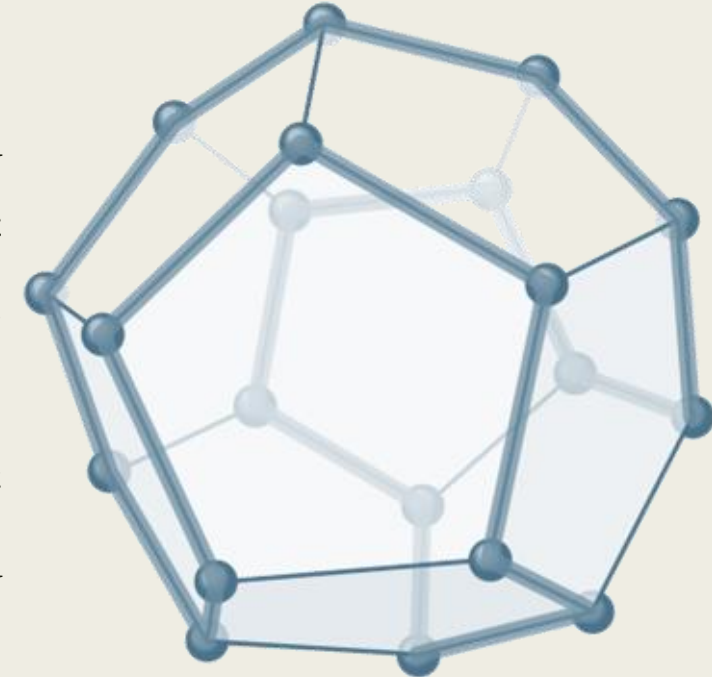
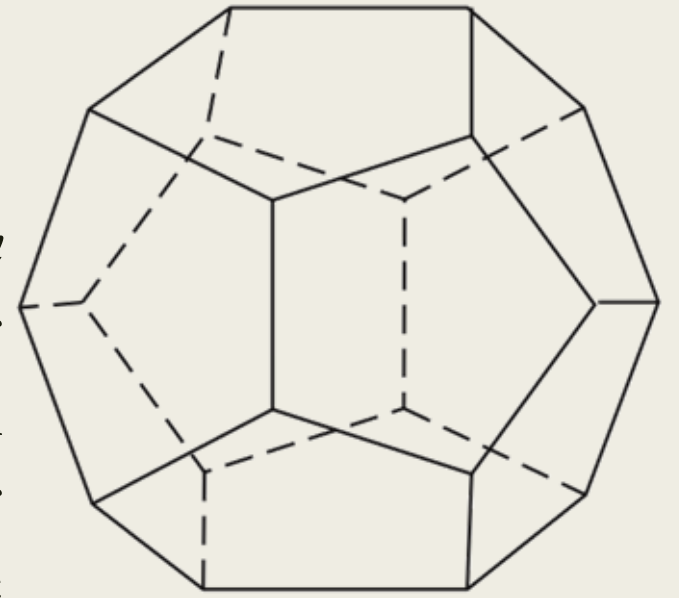
Q. Can we do the same for simple paths and circuits that contain every vertex of the graph exactly once?

DEFINITION. A simple path in a graph G that passes through every vertex exactly once is called a ***Hamilton path***, and a simple circuit in a graph G that passes through every vertex exactly once is called a ***Hamilton circuit***. That is, the simple path $x_0, x_1, \dots, x_{n-1}, x_n$ in the graph $G = (V, E)$ is a Hamilton path if $V = \{x_0, x_1, \dots, x_{n-1}, x_n\}$ and $x_i = x_j$ for $0 \leq i < j \leq n$, and the simple circuit $x_0, x_1, \dots, x_{n-1}, x_n, x_0$ (with $n > 0$) is a Hamilton circuit if $x_0, x_1, \dots, x_{n-1}, x_n$ is a Hamilton path.

Icosian puzzle

This terminology comes from a game, called the *Icosian puzzle*, invented in 1857 by the Irish mathematician Sir William Rowan Hamilton. It consisted of a wooden dodecahedron [a polyhedron with 12 regular pentagons as faces] with a peg at each vertex of the dodecahedron, and string.

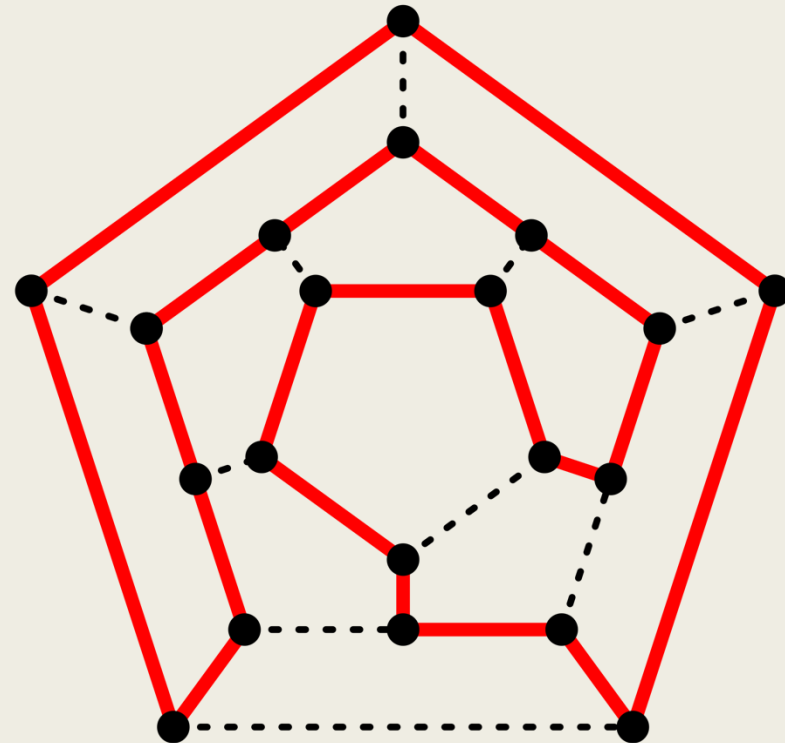
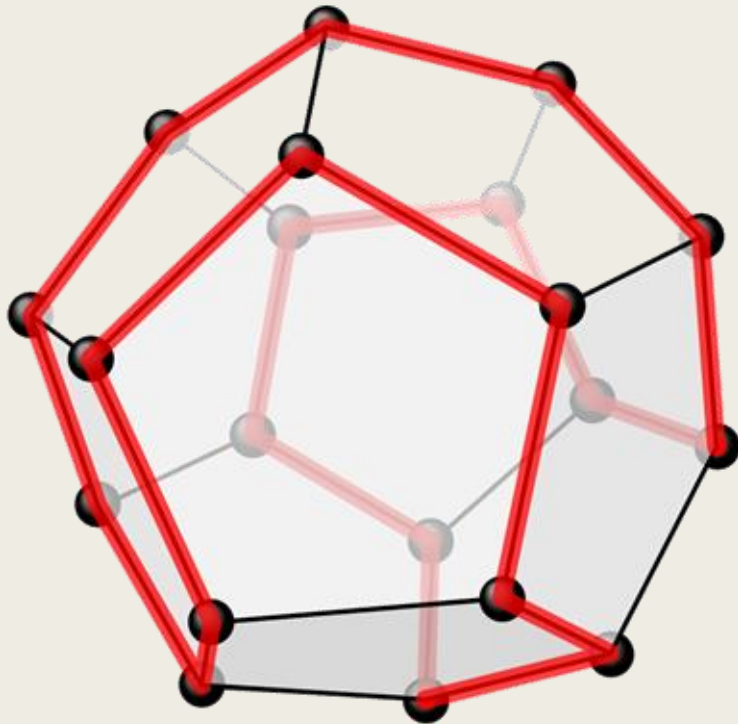
The 20 vertices of the dodecahedron were labeled with different cities in the world. The object of the puzzle was to start at a city and travel along the edges of the dodecahedron, visiting each of the other 19 cities exactly once, and end back at the first city. The circuit traveled was marked off using the string and pegs.



Icosian puzzle

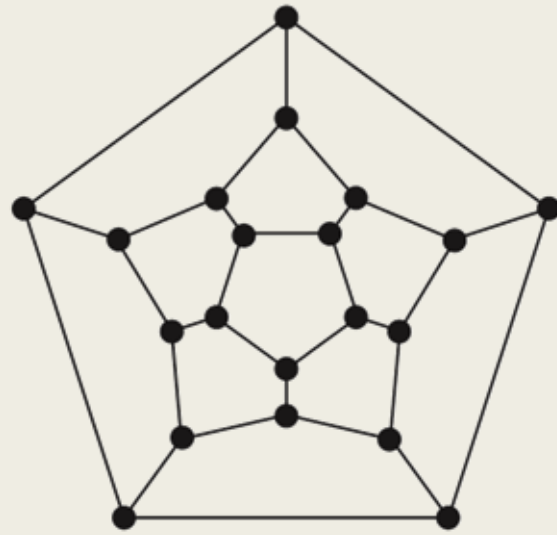
We will consider the equivalent question:

Q. Is there a circuit in the graph that passes through each vertex exactly once?

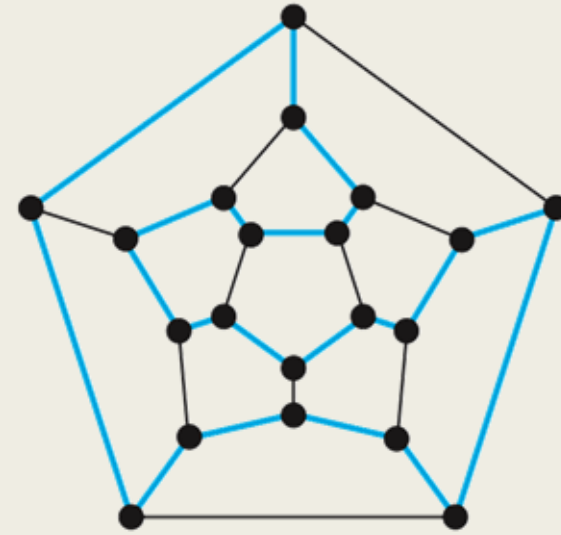


Icosian puzzle

This solves the puzzle because this graph is isomorphic to the graph consisting of the vertices and edges of the dodecahedron. A solution of Hamilton's puzzle is shown in (b).



(a)

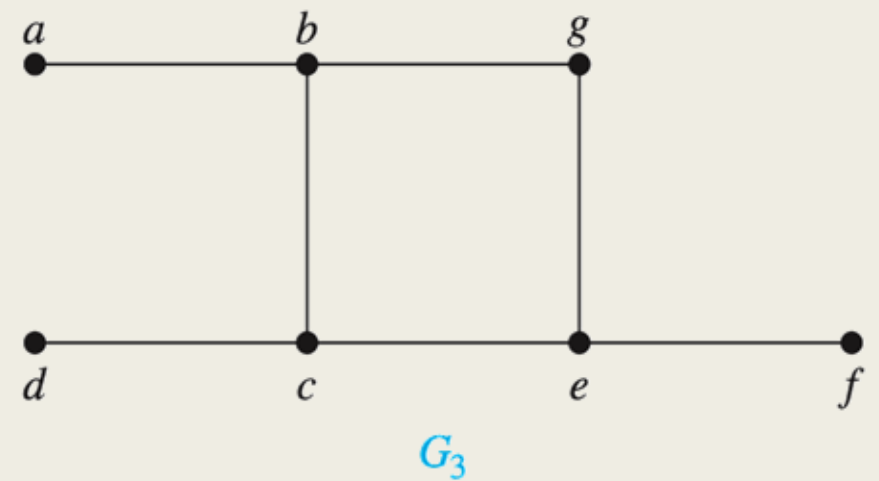
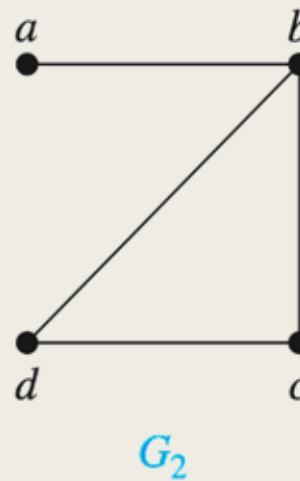
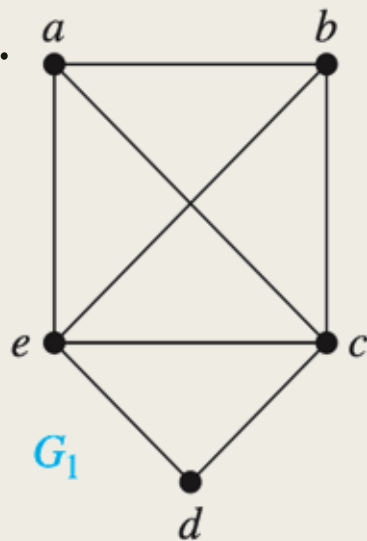


(b)

Hamilton Paths and Circuits: Example

EXAMPLE. Which of the following simple graphs have a Hamilton circuit or, if not, a Hamilton path?

Solution: G_1 has a Hamilton circuit: a, b, c, d, e, a . There is no Hamilton circuit in G_2 (this can be seen by noting that any circuit containing every vertex must contain the edge $\{a, b\}$ twice), but G_2 does have a Hamilton path, namely, a, b, c, d . G_3 has neither a Hamilton circuit nor a Hamilton path, because any path containing all vertices must contain one of the edges $\{a, b\}, \{e, f\}$, and $\{c, d\}$ more than once.



Conditions for the existence of Hamilton circuits

Q. Is there a simple way to determine whether a graph has a Hamilton circuit or path?

At first, it might seem that there should be an easy way to determine this, because there is a simple way to answer the similar question of whether a graph has an Euler circuit. Surprisingly, there are no known simple necessary and sufficient criteria for the existence of Hamilton circuits. However, many theorems are known that give sufficient conditions for the existence of Hamilton circuits.

Conditions for the existence of Hamilton circuits

Also, certain properties can be used to show that a graph has no Hamilton circuit. For instance,

- A graph with a vertex of degree one cannot have a Hamilton circuit, because in a Hamilton circuit, each vertex is incident with two edges in the circuit.
- If a vertex in the graph has degree two, then both edges that are incident with this vertex must be part of any Hamilton circuit.

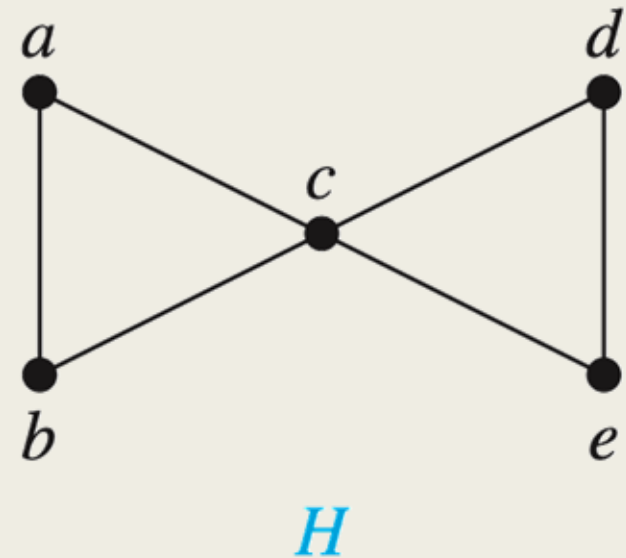
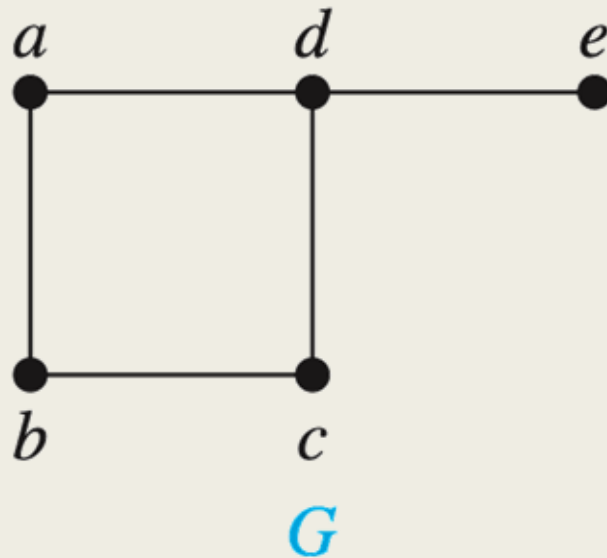
Also, note that when a Hamilton circuit is being constructed and this circuit has passed through a vertex, then all remaining edges incident with this vertex, other than the two used in the circuit, can be removed from consideration.

- Furthermore, a Hamilton circuit cannot contain a smaller circuit within it.

EXAMPLE. Show that neither graph has a Hamilton circuit.

Solution: There is no Hamilton circuit in G because G has a vertex of degree one, namely, e . Now consider H . Because the degrees of the vertices a, b, d , and e are all two, every edge incident with these vertices must be part of any Hamilton circuit.

It is now easy to see that no Hamilton circuit can exist in H , for any Hamilton circuit would have to contain four edges incident with c , which is impossible.



Conditions for the existence of Hamilton circuits: Example

EXAMPLE. Show that K_n has a Hamilton circuit whenever $n \geq 3$.

Solution: We can form a Hamilton circuit in K_n beginning at any vertex. Such a circuit can be built by visiting vertices in any order we choose, as long as the path begins and ends at the same vertex and visits each other vertex exactly once. This is possible because there are edges in K_n between any two vertices.

Conditions for the existence of Hamilton circuits

NOTE that the more edges a graph has, the more likely it is to have a Hamilton circuit.

Furthermore, adding edges (but not vertices) to a graph with a Hamilton circuit produces a graph with the same Hamilton circuit. So as we add edges to a graph, especially when we make sure to add edges to each vertex, we make it increasingly likely that a Hamilton circuit exists in this graph.

Consequently, we would expect there to be sufficient conditions for the existence of Hamilton circuits that depend on the degrees of vertices being sufficiently large.

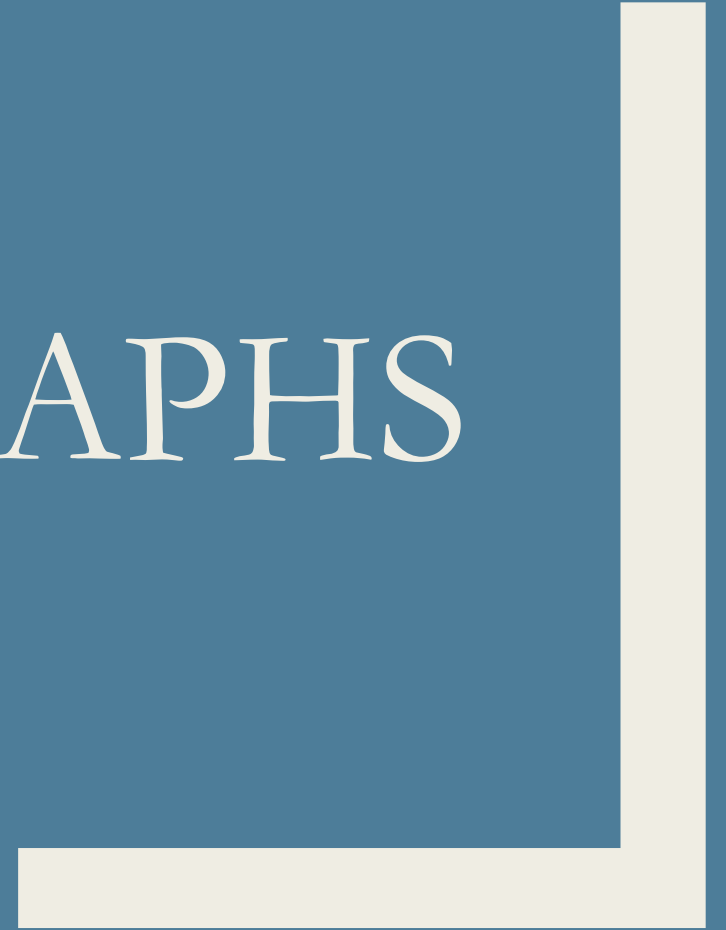
Conditions for the existence of Hamilton circuits

THEOREM (DIRAC'S THEOREM). If G is a simple graph with n vertices with $n \geq 3$ such that the degree of every vertex in G is at least $n/2$, then G has a Hamilton circuit.

THEOREM (ORE'S THEOREM). If G is a simple graph with n vertices with $n \geq 3$ such that $\deg(u) + \deg(v) \geq n$ for every pair of nonadjacent vertices u and v in G , then G has a Hamilton circuit.

Both Ore's theorem and Dirac's theorem provide sufficient conditions for a connected simple graph to have a Hamilton circuit. However, these theorems do not provide necessary conditions for the existence of a Hamilton circuit. For example, the graph C_5 has a Hamilton circuit but does not satisfy the hypotheses of either Ore's theorem or Dirac's theorem

PLANAR GRAPHS



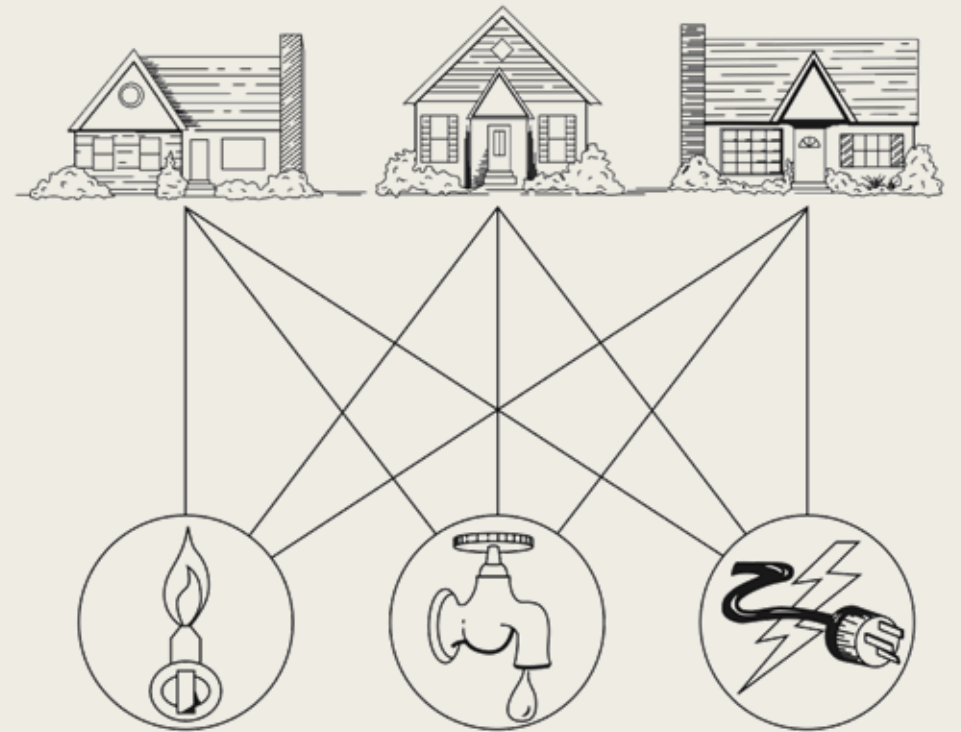
Planar Graphs

Consider the problem of joining 3 houses to each of 3 separate utilities.

Q. Is it possible to join these houses and utilities so that none of the connections cross?

This problem can be modeled using the complete bipartite graph $K_{3,3}$. The original question can be rephrased as:

Q. Can $K_{3,3}$ be drawn in the plane so that no two of its edges cross?



Planar Graphs

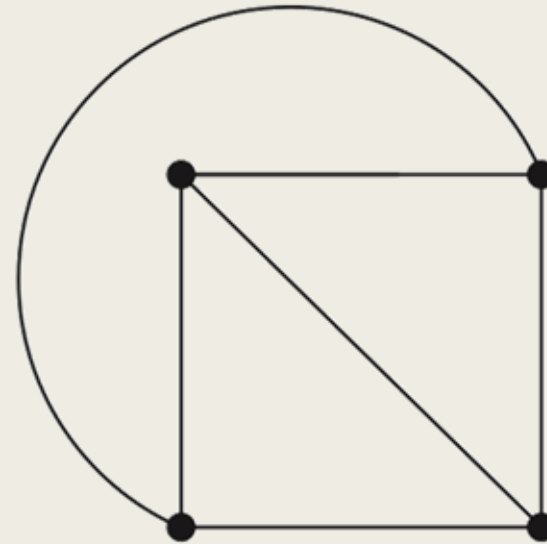
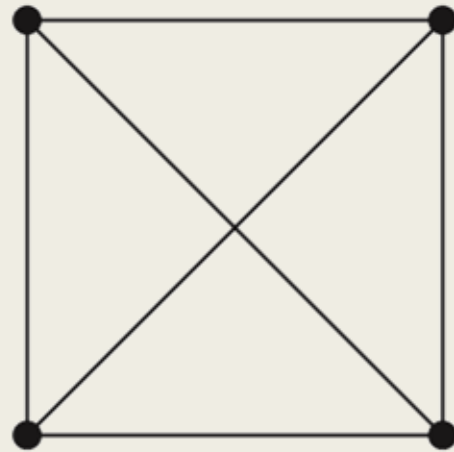
DEFINITION. A graph is called *planar* if it can be drawn in the plane without any edges crossing (where a crossing of edges is the intersection of the lines or arcs representing them at a point other than their common endpoint). Such a drawing is called a *planar representation* of the graph.

NOTE: A graph may be planar even if it is usually drawn with crossings, because it may be possible to draw it in a different way without crossings.

Planar Graphs: Example

EXAMPLE. Is K_4 planar?

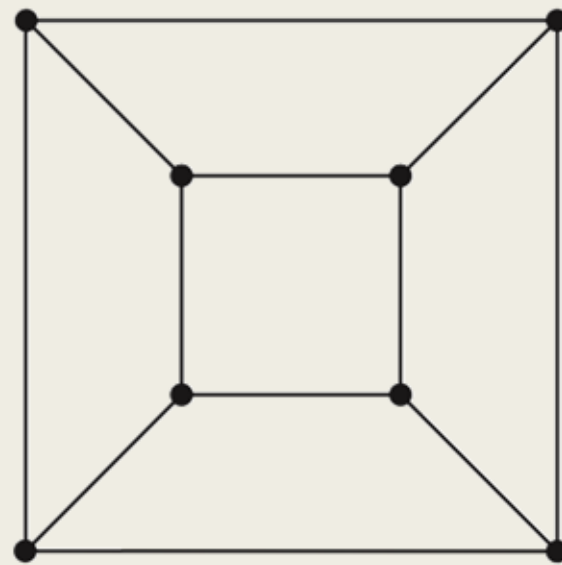
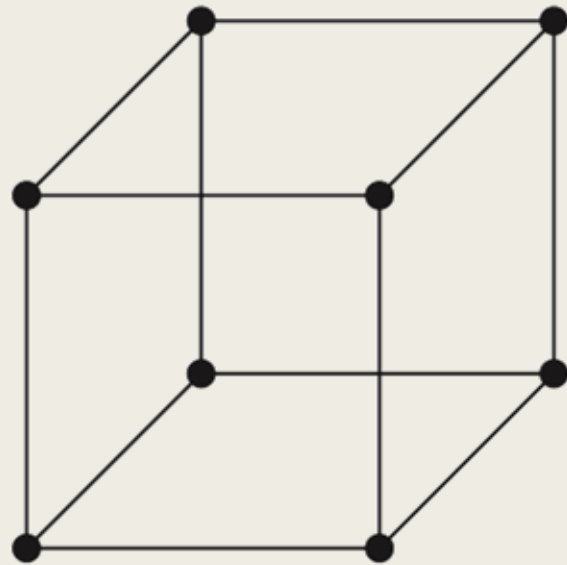
Answer: K_4 is planar because it can be drawn without crossings.



Planar Graphs: Example

EXAMPLE. Is Q_3 planar?

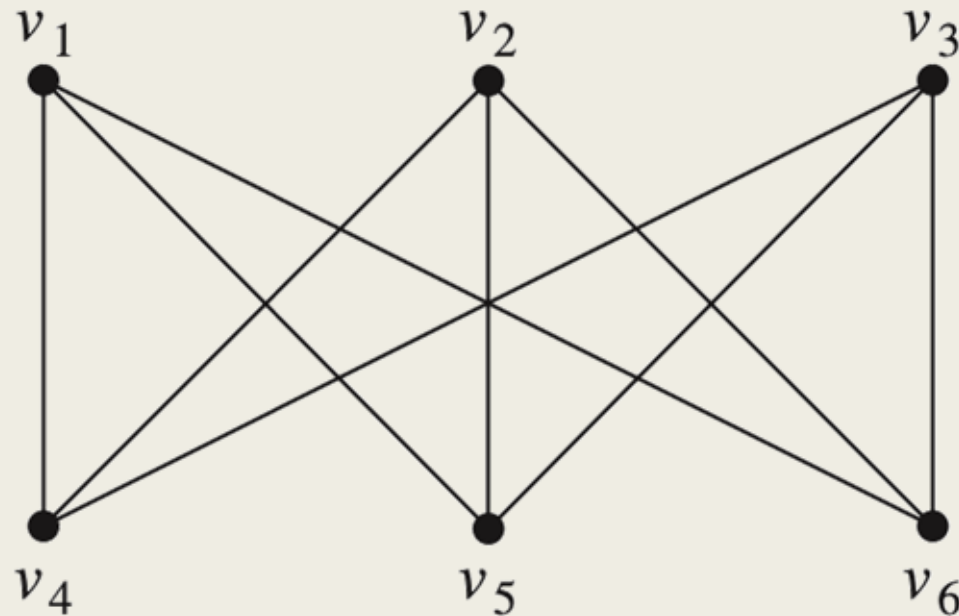
Answer: Q_3 is planar because it can be drawn without crossings.



Planar Graphs: Example

EXAMPLE. Is $K_{3,3}$ planar?

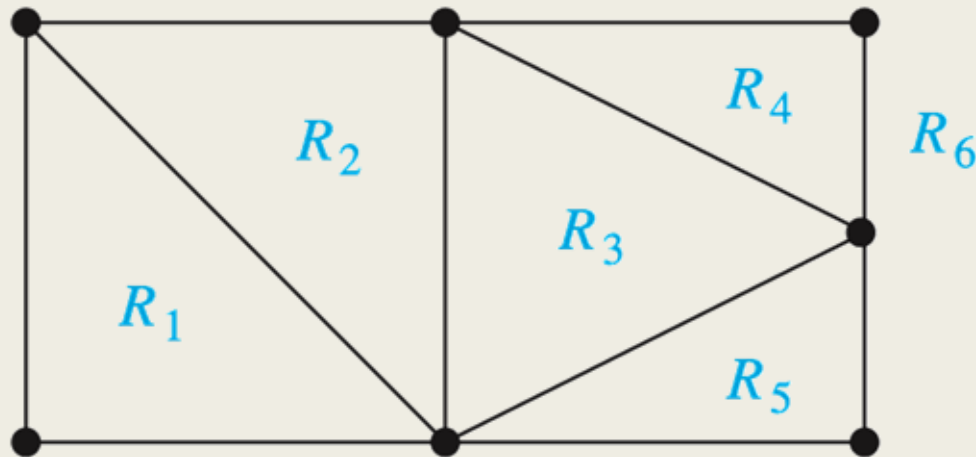
Answer: Any attempt to draw $K_{3,3}$ in the plane with no edges crossing is doomed. It follows that $K_{3,3}$ is not planar.



Euler's Formula

A planar representation of a graph splits the plane into **regions**, including an unbounded region.

For instance, the planar representation of the following graph splits the plane into six regions.



Euler's Formula

Euler showed that all planar representations of a graph split the plane into the same number of regions. He accomplished this by finding a relationship among the number of regions, the number of vertices, and the number of edges of a planar graph.

THEOREM (EULER'S FORMULA). Let G be a connected planar simple graph with e edges and v vertices. Let r be the number of regions in a planar representation of G . Then

$$r = e - v + 2$$

Euler's Formula: Example

EXAMPLE. Suppose that a connected planar simple graph has 20 vertices, each of degree 3. Into how many regions does a representation of this planar graph split the plane?

Solution: This graph has 20 vertices, each of degree 3, so $v = 20$. Because the sum of the degrees of the vertices, $3v = 3 \cdot 20 = 60$, is equal to twice the number of edges, $2e$, we have $2e = 60$, or $e = 30$. Consequently, from Euler's formula, the number of regions is

$$r = e - v + 2 = 30 - 20 + 2 = 12.$$

Euler's Formula

COROLLARY 1. If G is a connected planar simple graph with e edges and v vertices, where $v \geq 3$, then

$$e \leq 3v - 6$$

COROLLARY 2. If G is a connected planar simple graph, then G has a vertex of degree not exceeding 5.

Euler's Formula: Example

EXAMPLE. Show that K_5 is nonplanar using Corollary 1.

Solution: The graph K_5 has 5 vertices and 10 edges. However, the inequality

$$e \leq 3v - 6$$

is not satisfied for this graph because $e = 10$ and $3v - 6 = 9$.

$$10 \not\leq 9$$

Therefore, K_5 is not planar.

Euler's Formula

It was previously shown that $K_{3,3}$ is not planar.

NOTE, however, that this graph has 6 vertices and 9 edges. This means that the inequality

$$e = 9 \leq 12 = 3 \cdot 6 - 6$$

is satisfied. Consequently, the fact that the inequality $e \leq 3v - 6$ is satisfied **does not** imply that a graph is planar. However, the following corollary of Theorem 1 can be used to show that $K_{3,3}$ is nonplanar.

COROLLARY 3. If a connected planar simple graph has e edges and v vertices with $v \geq 3$ and no circuits of length three, then

$$e \leq 2v - 4$$

Euler's Formula: Example

EXAMPLE. Use Corollary 3 to show that $K_{3,3}$ is nonplanar.

Solution: Because $K_{3,3}$ has no circuits of length 3 (this is easy to see because it is bipartite), Corollary 3 can be used. $K_{3,3}$ has 6 vertices and 9 edges. Because $e = 9$ and $2v - 4 = 8$, Corollary 3 shows that $K_{3,3}$ is nonplanar.

$$9 \not\leq 8$$

Kuratowski's Theorem

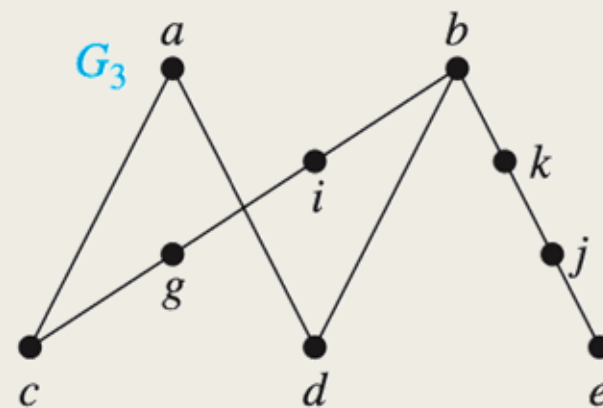
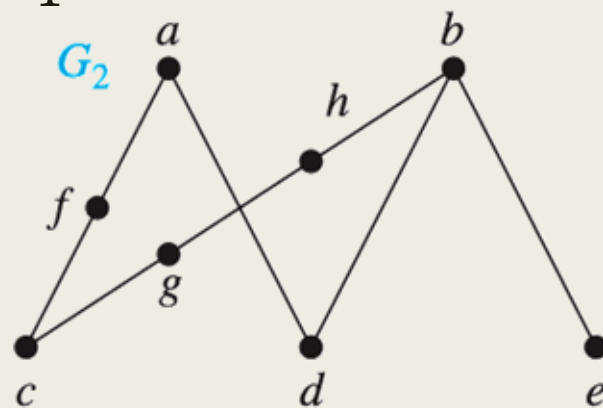
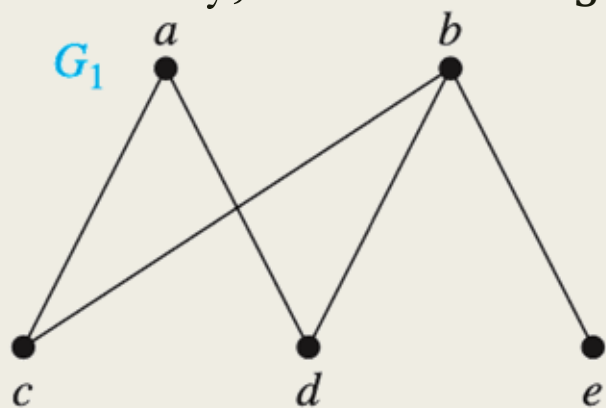
We have seen that $K_{3,3}$ and K_5 are not planar. Clearly, a graph is not planar if it contains either of these two graphs as a subgraph. Surprisingly, all nonplanar graphs must contain a subgraph that can be obtained from $K_{3,3}$ or K_5 using certain permitted operations.

- If a graph is planar, so will be any graph obtained by removing an edge $\{u, v\}$ and adding a new vertex w together with edges $\{u, w\}$ and $\{w, v\}$. Such an operation is called an *elementary subdivision*.
- The graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are called *homeomorphic* if they can be obtained from the same graph by a sequence of elementary subdivisions.

EXAMPLE. Show that the graphs G_1, G_2 , and G_3 are all homeomorphic.

Solution: These three graphs are homeomorphic because all three can be obtained from G_1 by elementary subdivisions. G_1 can be obtained from itself by an empty sequence of elementary subdivisions. To obtain G_2 from G_1 we can use this sequence of elementary subdivisions:

- (i) remove the edge $\{a, c\}$, add the vertex f , and add the edges $\{a, f\}$ and $\{f, c\}$;
 - (ii) remove the edge $\{b, c\}$, add the vertex g , and add the edges $\{b, g\}$ and $\{g, c\}$;
and
 - (iii) remove the edge $\{b, g\}$, add the vertex h , and add the edges $\{g, h\}$ and $\{b, h\}$.
- Similarly, we obtain G_3 from G_1 .



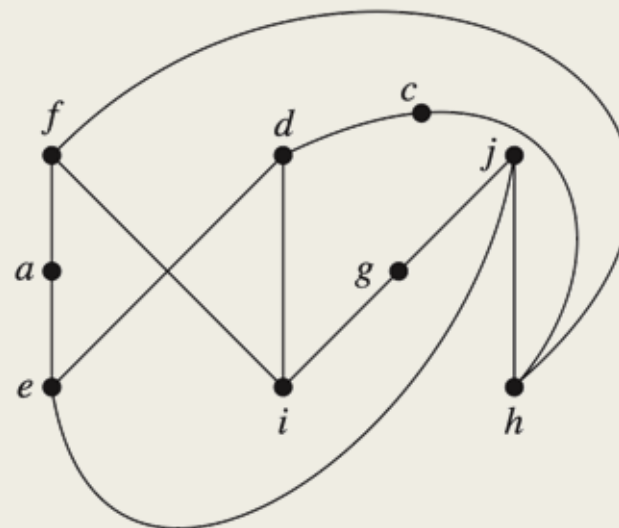
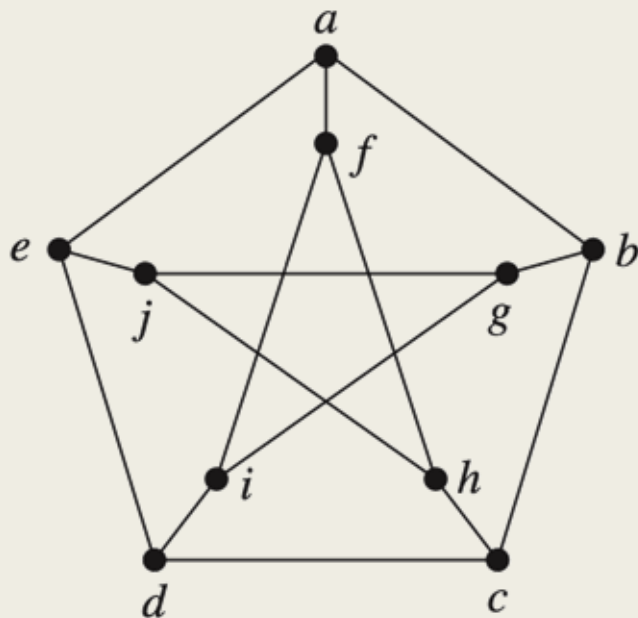
Kuratowski's Theorem

The Polish mathematician Kazimierz Kuratowski established Theorem 2 in 1930, which characterizes planar graphs using the concept of graph homeomorphism.

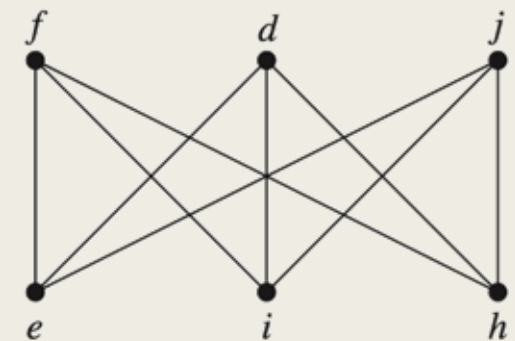
THEOREM. A graph is nonplanar if and only if it contains a subgraph homeomorphic to $K_{3,3}$ or K_5 .

EXAMPLE. Is the Petersen graph planar?

Solution: The subgraph H of the Petersen graph obtained by deleting b and the three edges that have b as an endpoint, shown in (b), is homeomorphic to $K_{3,3}$, with vertex sets $\{f, d, j\}$ and $\{e, i, h\}$, because it can be obtained by a sequence of elementary subdivisions, deleting $\{d, h\}$ and adding $\{c, h\}$ and $\{c, d\}$, deleting $\{e, f\}$ and adding $\{a, e\}$ and $\{a, f\}$, and deleting $\{i, j\}$ and adding $\{g, i\}$ and $\{g, j\}$. Hence, the Petersen graph is not planar.



(b) H



(c) $K_{3,3}$

TREES



A connected graph that contains no simple circuits is called a *tree*.

Trees were used as long ago as 1857, when the English mathematician Arthur Cayley used them to count certain types of chemical compounds. Since that time, trees have been employed to solve problems in a wide variety of disciplines.

Trees are particularly useful in computer science, where they are employed in a wide range of algorithms.

Trees are particularly useful in computer science, where they are employed in a wide range of algorithms. For instance, trees are used

- ✓ to construct efficient algorithms for locating items in a list.
- ✓ in algorithms, such as Huffman coding, that construct efficient codes saving costs in data transmission and storage.
- ✓ to study games such as checkers and chess and can help determine winning strategies for playing these games.
- ✓ to model procedures carried out using a sequence of decisions.

Constructing these models can help determine the computational complexity of algorithms based on a sequence of decisions, such as sorting algorithms.

For example, *family trees* are graphs that represent genealogical charts. Family trees use vertices to represent the members of a family and edges to represent parent-child relationships. The family tree of the male members of the Bernoulli family of Swiss mathematicians is shown here.

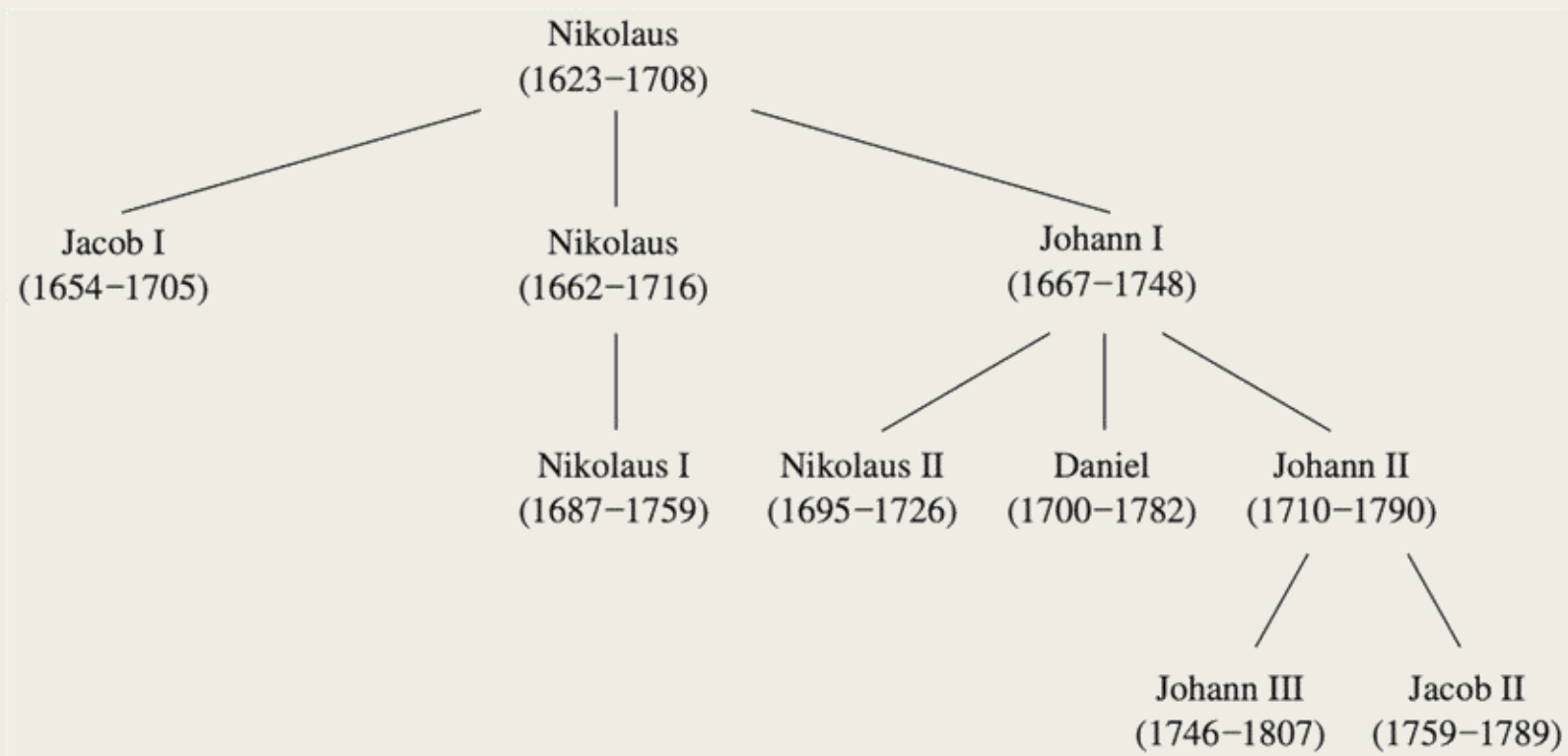


FIGURE 1 The Bernoulli family of mathematicians.

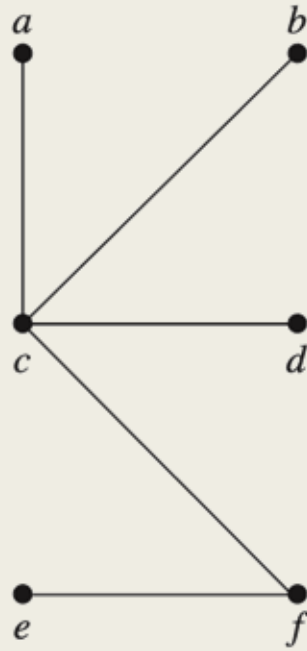
DEFINITION. A *tree* is a connected undirected graph with no simple circuits.

Because a tree cannot have a simple circuit, a tree cannot contain multiple edges or loops. Therefore, **any tree must be a simple graph.**

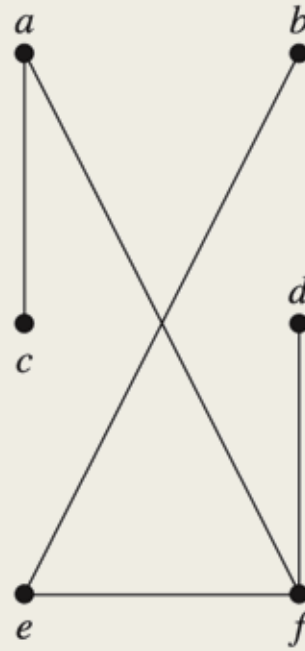
THEOREM 1. An undirected graph is a tree if and only if there is a unique simple path between any two of its vertices.

EXAMPLE. Which of the following graphs are trees?

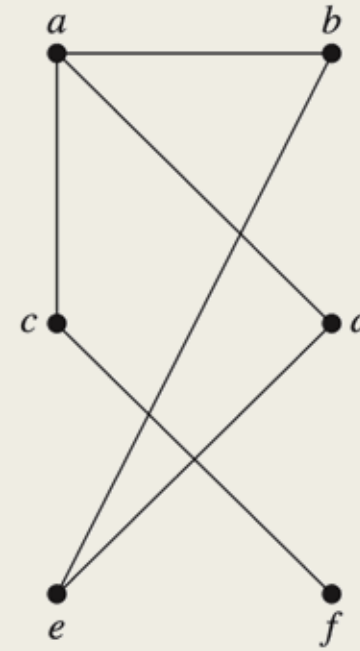
Solution: G_1 and G_2 are trees, because both are connected graphs with no simple circuits. G_3 is not a tree because e, b, a, d, e is a simple circuit in this graph. Finally, G_4 is not a tree because it is not connected.



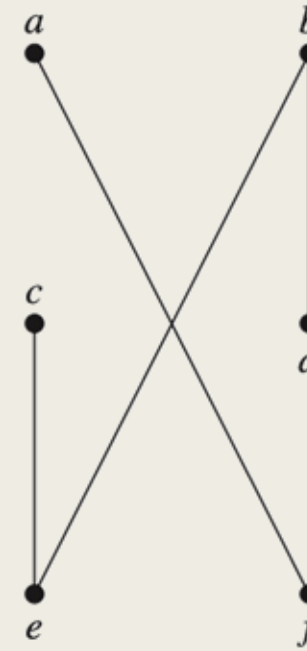
G_1



G_2



G_3



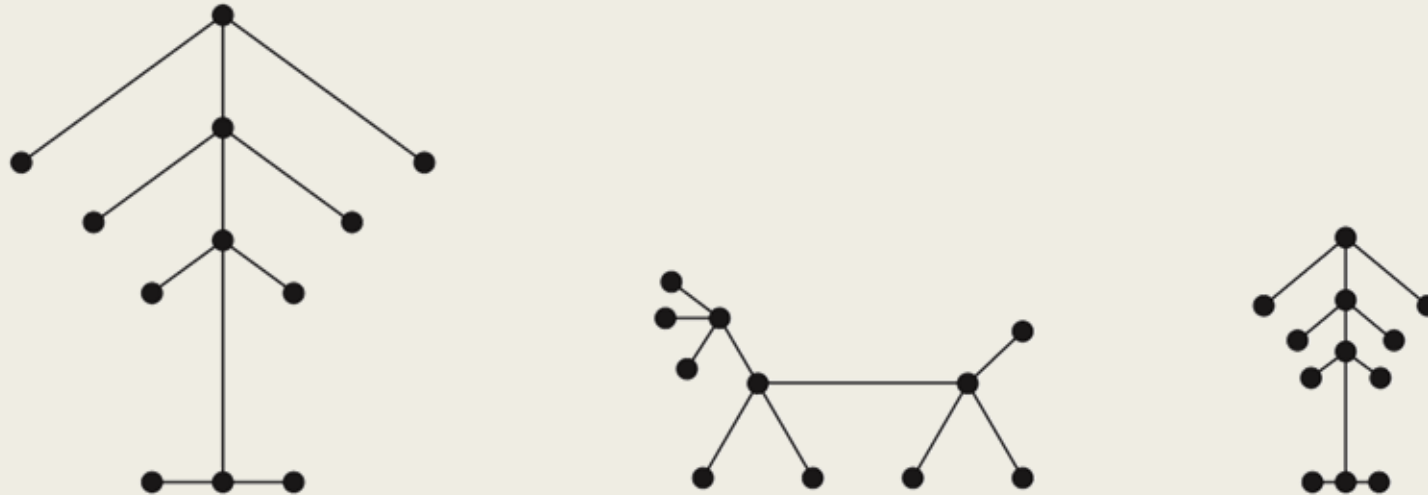
G_4

Any connected graph that contains no simple circuits is a tree.

Q. What about graphs containing no simple circuits that are not necessarily connected?

These graphs are called ***forests*** and have the property that each of their connected components is a tree.

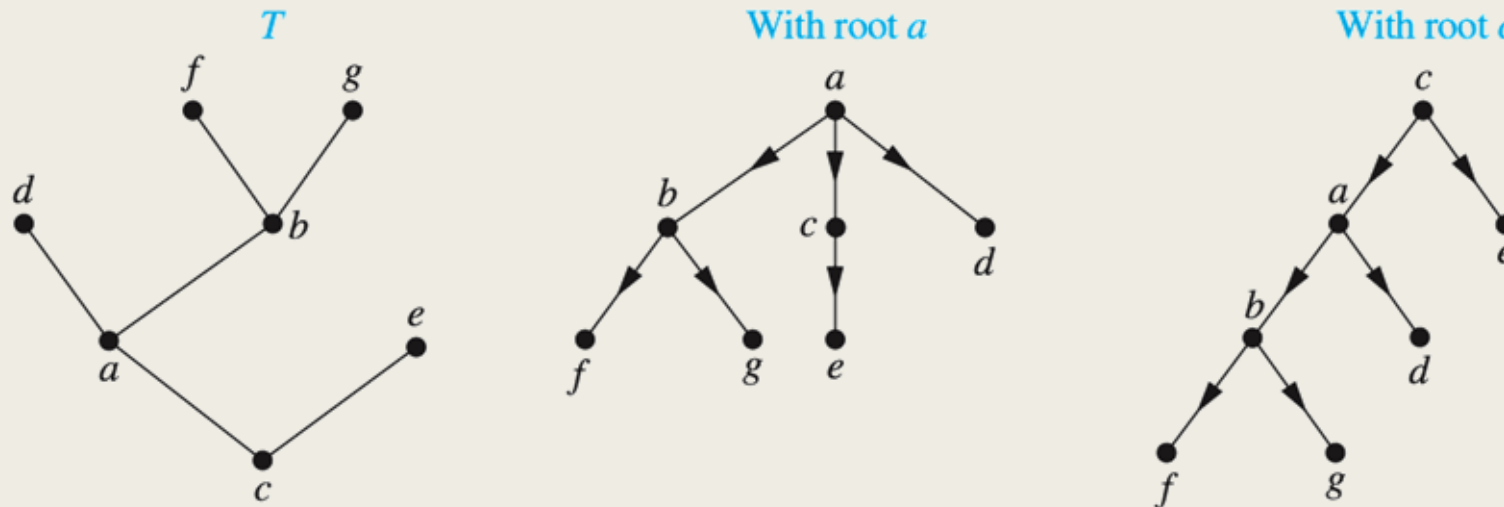
Trees are often defined as undirected graphs with the property that there is a unique simple path between every pair of vertices.



DEFINITION. A *rooted tree* is a tree in which one vertex has been designated as the root and every edge is directed away from the root.

Rooted trees can also be defined recursively. We can change an unrooted tree into a rooted tree by choosing any vertex as the root. Note that different choices of the root produce different rooted trees.

We usually draw a rooted tree with its root at the top of the graph. The arrows indicating the directions of the edges in a rooted tree can be omitted, because the choice of root determines the directions of the edges.

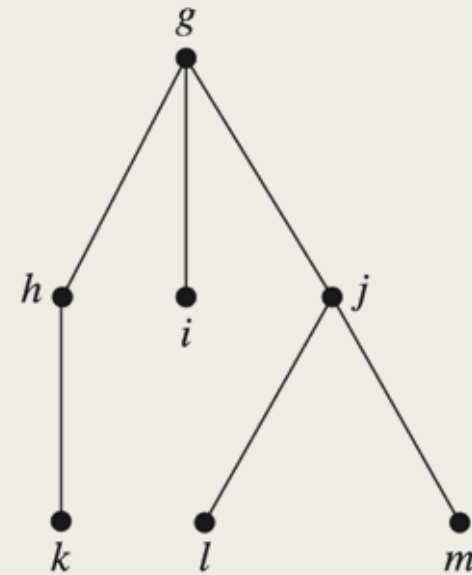
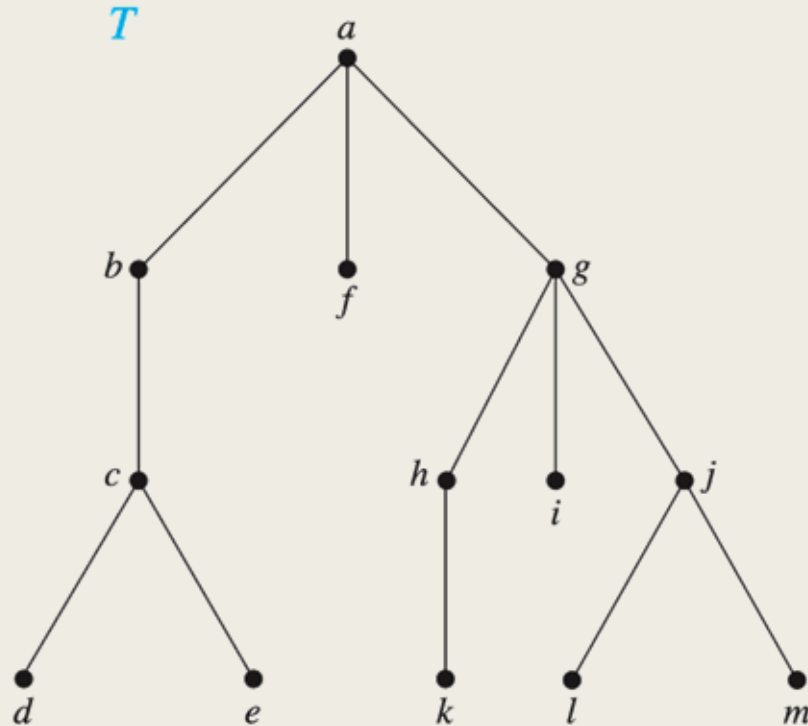


The terminology for trees has botanical and genealogical origins. Suppose that T is a rooted tree.

- If v is a vertex in T other than the root, the **parent** of v is the unique vertex u such that there is a directed edge from u to v .
- When u is the parent of v , v is called a **child** of u .
- Vertices with the same parent are called **siblings**.
- The **ancestors** of a vertex other than the root are the vertices in the path from the root to this vertex, excluding the vertex itself and including the root (that is, its parent, its parent's parent, and so on, until the root is reached).
- The **descendants** of a vertex v are those vertices that have v as an ancestor.
- A vertex of a rooted tree is called a **leaf** if it has no children. Vertices that have children are called **internal vertices**. The root is an internal vertex unless it is the only vertex in the graph, in which case it is a leaf.
- If a is a vertex in a tree, the **subtree** with a as its root is the subgraph of the tree consisting of a and its descendants and all edges incident to these descendants.

EXAMPLE. In the following rooted tree T (with root a), find the parent of c , the children of g , the siblings of h , all ancestors of e , all descendants of b , all internal vertices, and all leaves. What is the subtree rooted at g ?

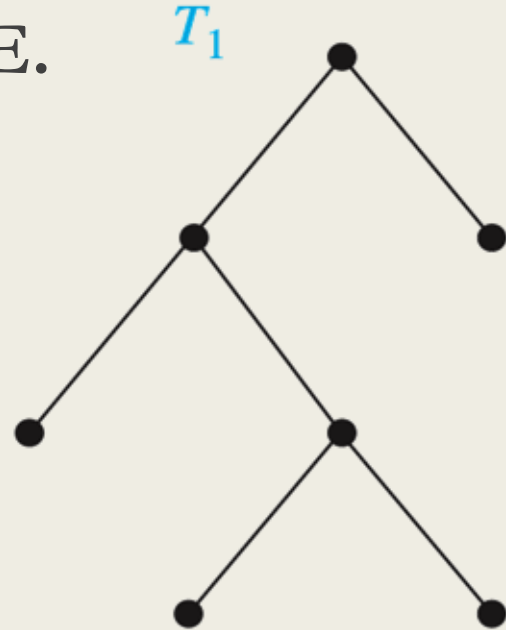
Solution: The parent of c is b . The children of g are h , i , and j . The siblings of h are i and j . The ancestors of e are c , b , and a . The descendants of b are c , d , and e . The internal vertices are a , b , c , g , h , and j . The leaves are d , e , f , i , k , l , and m .



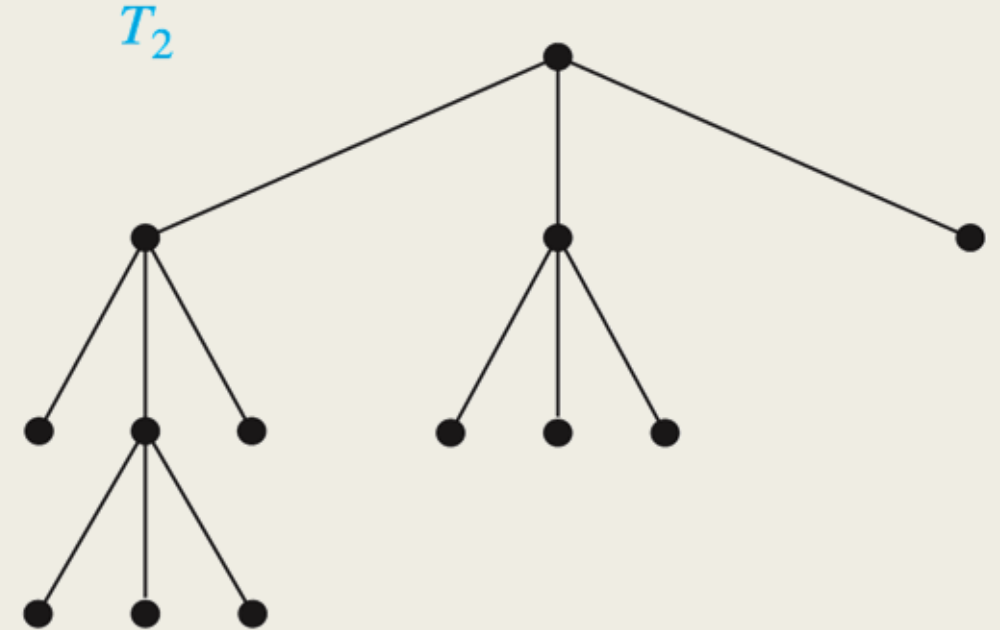
The rooted subtree at g

DEFINITION. A rooted tree is called an *m -ary tree* if every internal vertex has no more than m children. The tree is called a *full m -ary tree* if every internal vertex has exactly m children. An m -ary tree with $m = 2$ is called a *binary tree*.

EXAMPLE.

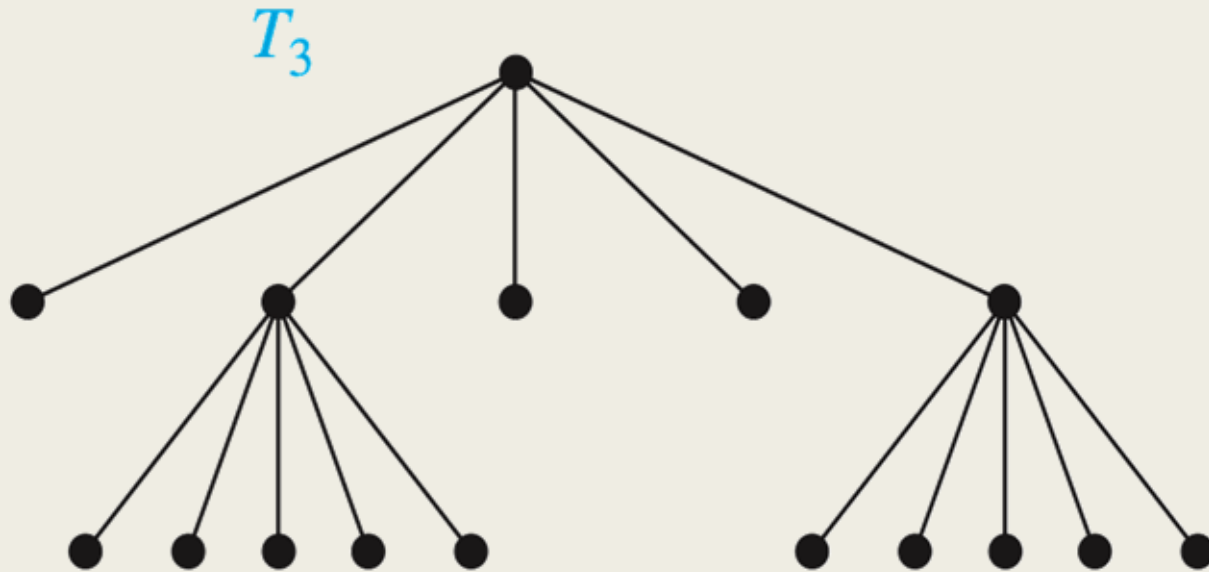


T_1 is a full binary tree because each of its internal vertices has 2 children.

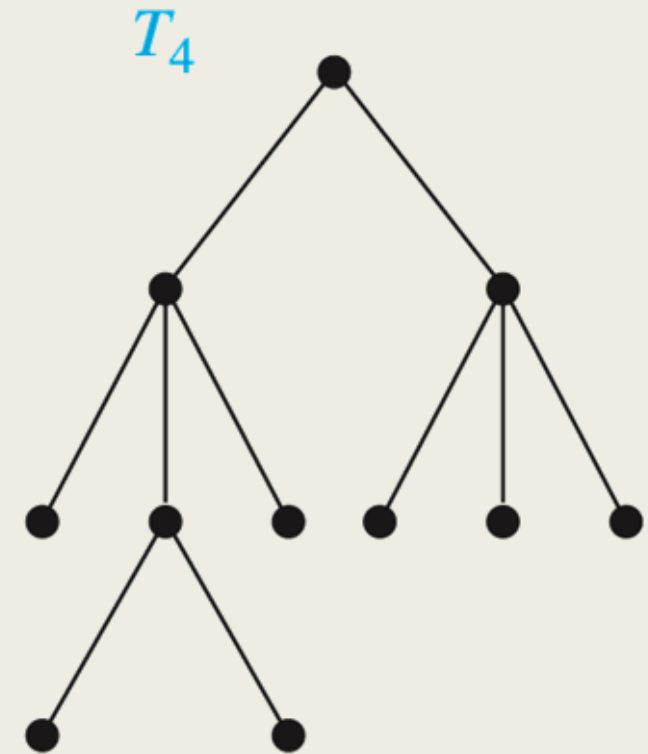


T_2 is a full 3-ary tree because each of its internal vertices has 3 children.

EXAMPLE.



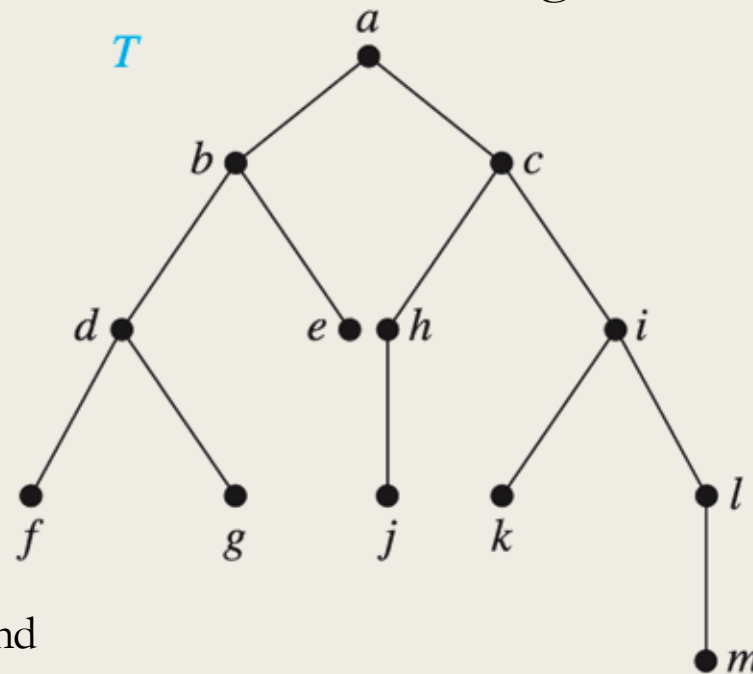
In T_3 each internal vertex has 5 children, so T_3 is a full 5-ary tree.



T_4 is not a full m -ary tree for any m because some of its internal vertices have 2 children and others have 3 children.

An **ordered rooted tree** is a rooted tree where the children of each internal vertex are ordered. Ordered rooted trees are drawn so that the children of each internal vertex are shown in order from left to right.

In an ordered binary tree (usually called just a **binary tree**), if an internal vertex has two children, the first child is called the **left child** and the second child is called the **right child**. The tree rooted at the left child of a vertex is called the **left subtree** of this vertex, and the tree rooted at the right child of a vertex is called the **right subtree** of the vertex.

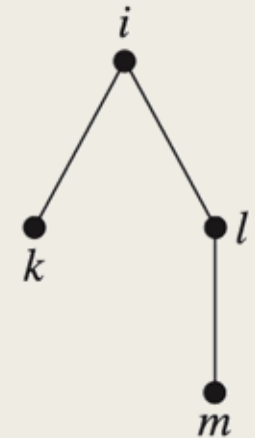


A binary tree T and left and right subtrees of the vertex c

(a)



(b)



(c)

Properties of Trees

THEOREM 2. A tree with n vertices has $n - 1$ edges.

Recall that a tree is a connected undirected graph with no simple circuits. So, when G is an undirected graph with n vertices, Theorem 2 tells us that the two conditions

- (i) G is connected and (ii) G has no simple circuits, imply
- (iii) G has $n - 1$ edges.

Counting vertices in full m -ary trees

We will use n to denote the number of vertices in a tree.

THEOREM 3. A full m -ary tree with i internal vertices contains $n = m \cdot i + 1$ vertices.

THEOREM 4. A full m -ary tree with

(i) n vertices has $i = \frac{n-1}{m}$ internal vertices and $l = \frac{(m-1)n+1}{m}$ leaves,

(ii) i internal vertices has $n = mi + 1$ vertices and $l = (m - 1)i + 1$ leaves,

(iii) l leaves has $n = \frac{ml-1}{m-1}$ vertices and $i = \frac{l-1}{m-1}$ internal vertices.

EXAMPLE. Suppose that someone starts a chain letter. Each person who receives the letter is asked to send it on to four other people. Some people do this, but others do not send any letters. How many people have seen the letter, including the first person, if no one receives more than one letter and if the chain letter ends after there have been 100 people who read it but did not send it out? How many people sent out the letter?

Solution: The chain letter can be represented using a 4-ary tree. The internal vertices correspond to people who sent out the letter, and the leaves correspond to people who did not send it out. Because 100 people did not send out the letter, the number of leaves in this rooted tree is $l = 100$. Hence, part (iii) of Theorem 4 shows that the number of people who have seen the letter is $n = (4 \cdot 100 - 1)/(4 - 1) = 133$. Also, the number of internal vertices is $133 - 100 = 33$, so 33 people sent out the letter.

Balanced m -ary Trees

It is often desirable to use rooted trees that are “balanced” so that the subtrees at each vertex contain paths of approximately the same length. Some definitions will make this concept clear.

- The *level* of a vertex v in a rooted tree is the length of the unique path from the root to this vertex. The level of the root is defined to be zero.
- The *height* of a rooted tree is the maximum of the levels of vertices. In other words, the height of a rooted tree is the length of the longest path from the root to any vertex.

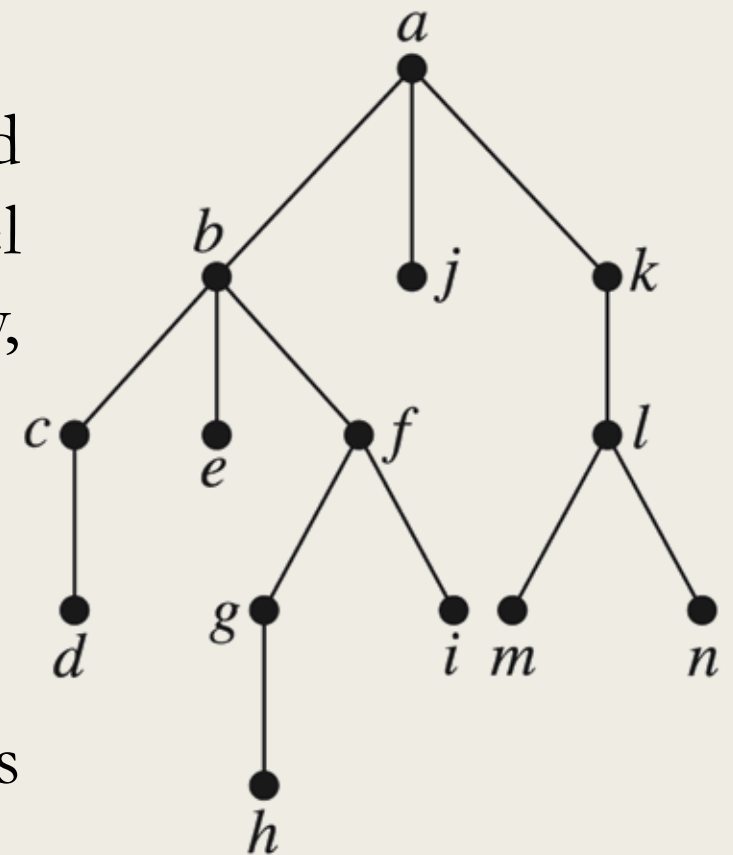
Balanced m -ary Trees: Example

EXAMPLE. Find the level of each vertex in the following rooted tree. What is the height of this tree?

Answer: The root a is at level 0. Vertices b, j , and k are at level 1. Vertices c, e, f , and l are at level 2. Vertices d, g, i, m , and n are at level 3. Finally, vertex h is at level 4.

Because the largest level of any vertex is 4, this tree has height 4.

Remark: A rooted m -ary tree of height h is *balanced* if all leaves are at levels h or $h - 1$.



A bound for the number of leaves in an m -ary tree

THEOREM 5. There are at most m^h leaves in an m -ary tree of height h .

COROLLARY 1. If an m -ary tree of height h has l leaves, then

$$h \geq \lceil \log_m l \rceil$$

If the m -ary tree is full and balanced, then

$$h = \lceil \log_m l \rceil$$

We are using the ceiling function here. Recall that $\lceil x \rceil$ is the smallest integer greater than or equal to x .

Tree Traversal

Ordered rooted trees are often used to store information. We need procedures for visiting each vertex of an ordered rooted tree to access data.

Procedures for systematically visiting every vertex of an ordered rooted tree are called ***traversal algorithms***. We will describe three of the most commonly used such algorithms,

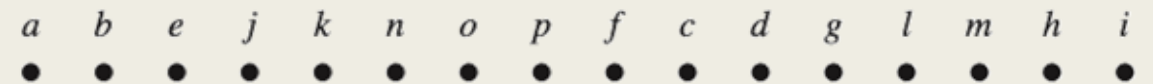
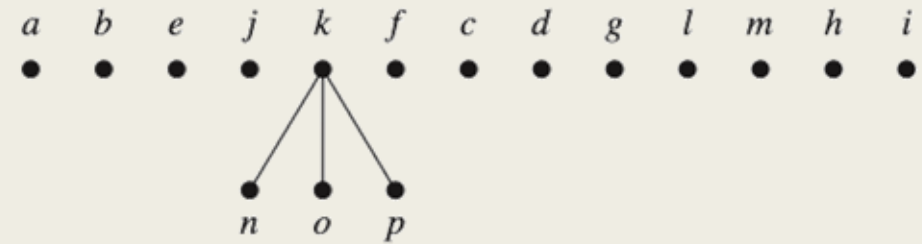
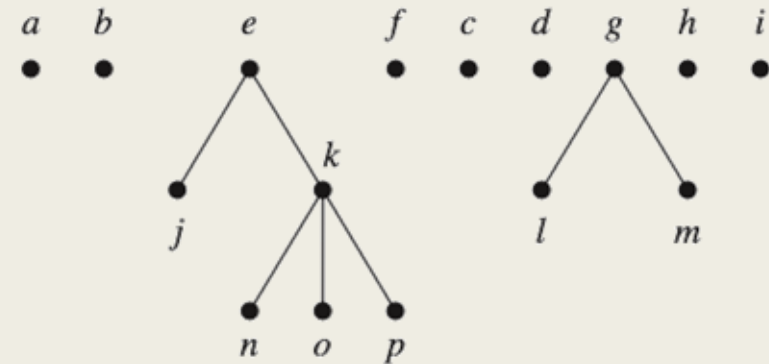
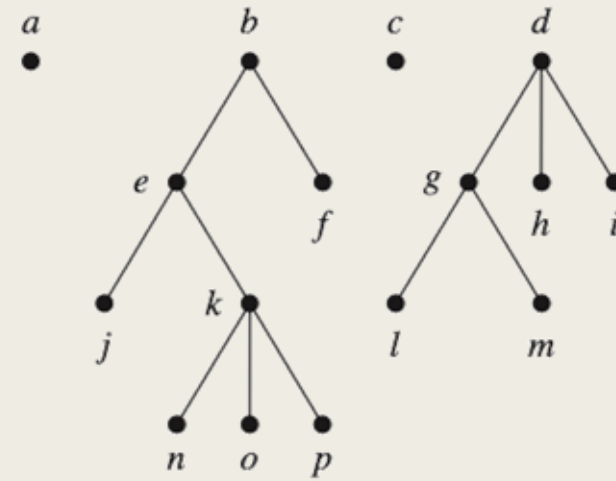
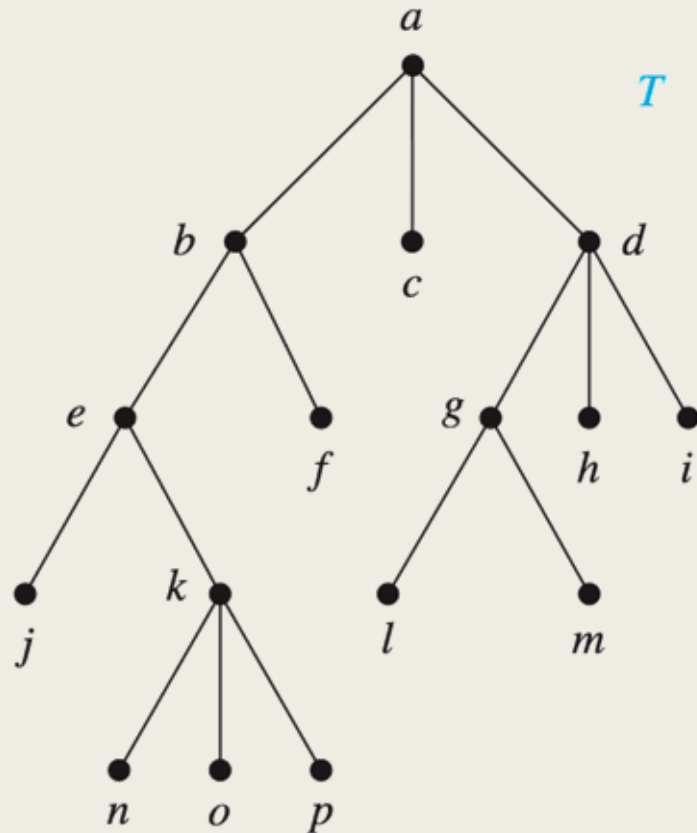
1. **preorder traversal**,
2. **inorder traversal**, and
3. **postorder traversal**.

Each of these algorithms can be defined recursively.

Traversal Algorithms

DEFINITION. Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *preorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right in T . The *preorder traversal* begins by visiting r . It continues by traversing T_1 in preorder, then T_2 in preorder, and so on, until T_n is traversed in preorder.

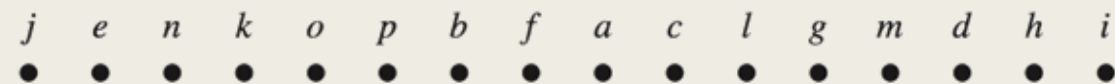
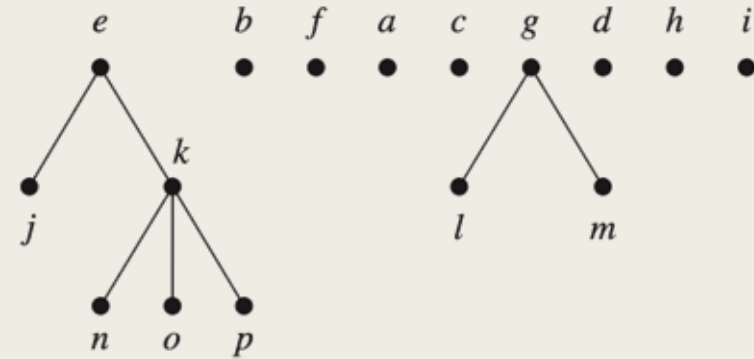
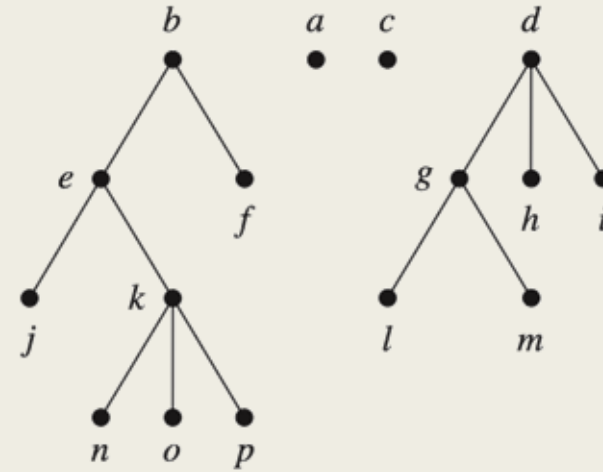
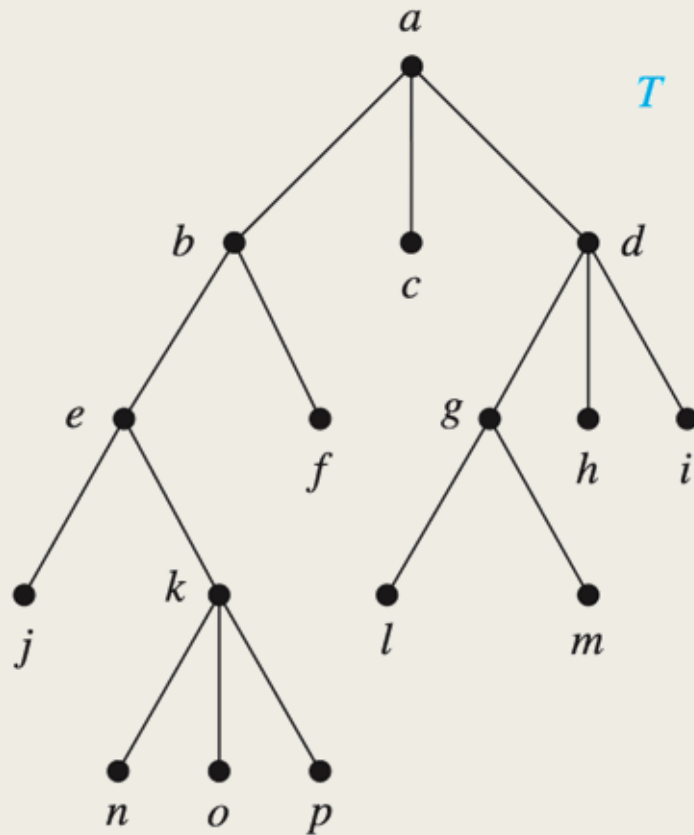
EXAMPLE. In which order does a preorder traversal visit the vertices in the ordered rooted tree T ?



Traversal Algorithms

DEFINITION. Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *inorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right. The *inorder traversal* begins by traversing T_1 in inorder, then visiting r . It continues by traversing T_2 in inorder, then T_3 in inorder, . . . , and finally T_n in inorder.

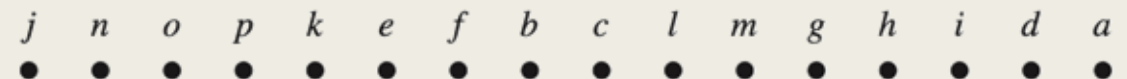
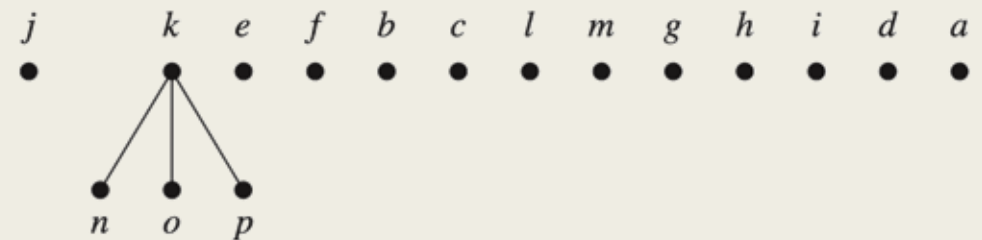
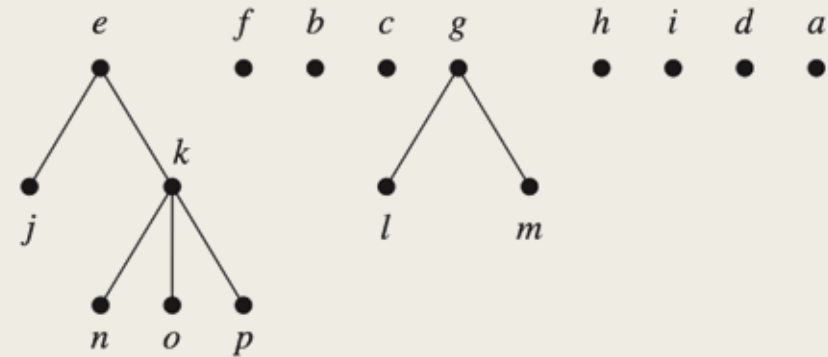
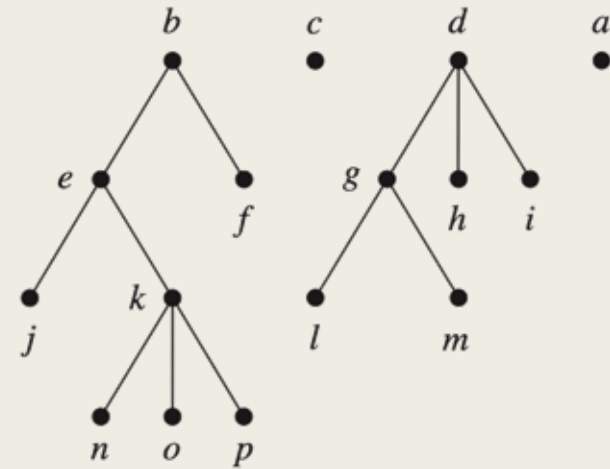
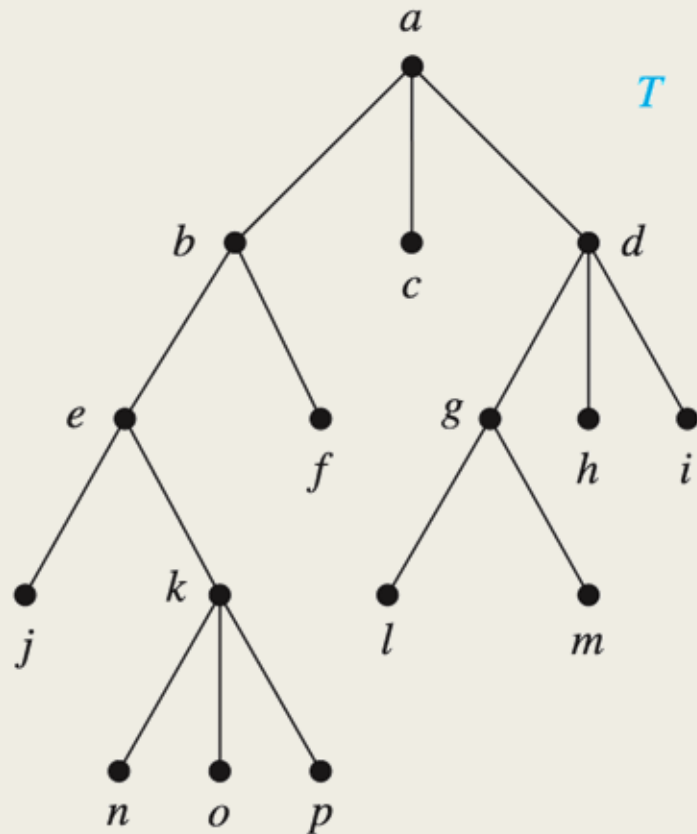
EXAMPLE. In which order does an inorder traversal visit the vertices of the ordered rooted tree T ?



Traversal Algorithms

DEFINITION. Let T be an ordered rooted tree with root r . If T consists only of r , then r is the *postorder traversal* of T . Otherwise, suppose that $T_1, T_2, \dots, T_n$ are the subtrees at r from left to right. The *postorder traversal* begins by traversing T_1 in postorder, then T_2 in postorder, $\dots$, then T_n in postorder, and ends by visiting r .

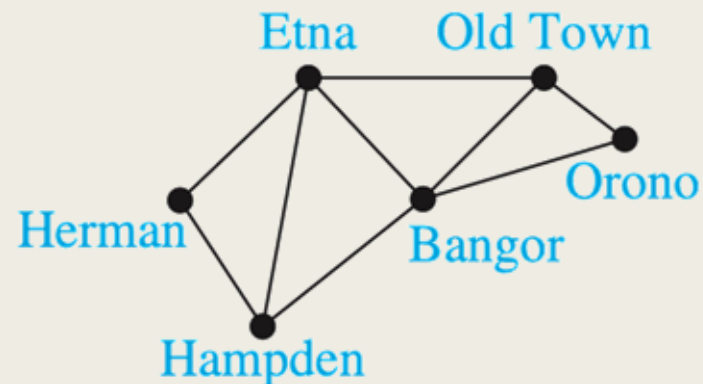
EXAMPLE. In which order does a postorder traversal visit the vertices of the ordered rooted tree T ?



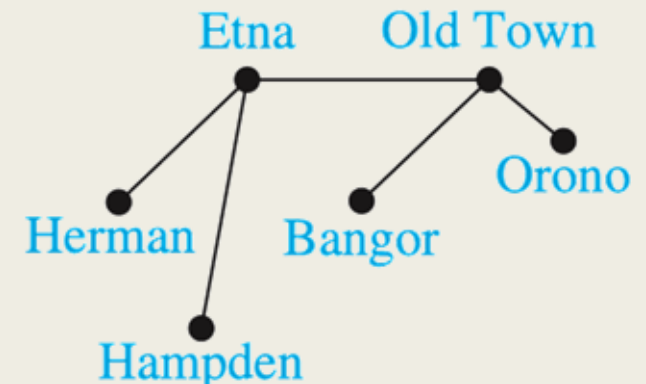
Spanning Trees

EXAMPLE. Consider the system of roads in Maine represented by the simple graph shown in (a). The only way the roads can be kept open in the winter is by frequently plowing them. The highway department wants to plow the fewest roads so that there will always be cleared roads connecting any two towns. How can this be done?

At least 5 roads must be plowed to ensure that there is a path between any 2 towns. (b) shows one such set of roads. Note that the subgraph representing these roads is a tree, because it is connected and contains 6 vertices and 5 edges.



(a)



(b)

Spanning Trees

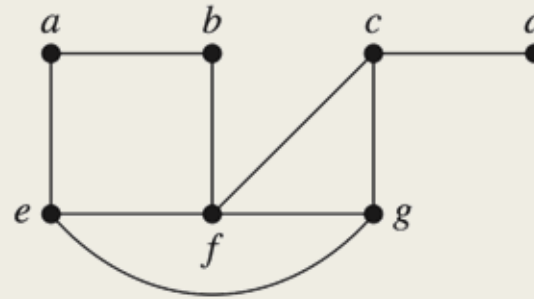
The previous problem was solved with a connected subgraph with the minimum number of edges containing all vertices of the original simple graph. Such a graph must be a tree.

DEFINITION. Let G be a simple graph. A *spanning tree* of G is a subgraph of G that is a tree containing every vertex of G .

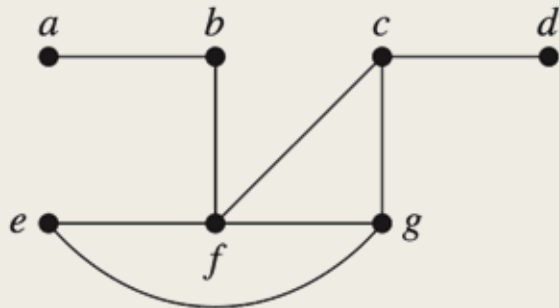
A simple graph with a spanning tree must be connected, because there is a path in the spanning tree between any two vertices. The converse is also true; that is,

every connected simple graph has a spanning tree

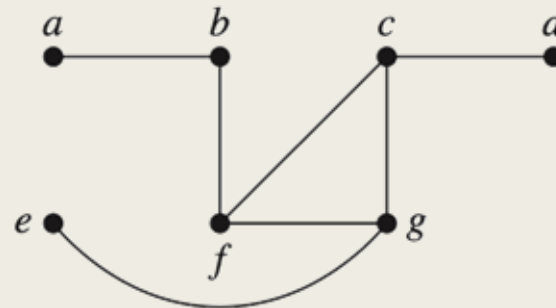
EXAMPLE. Find a spanning tree of the simple graph G .



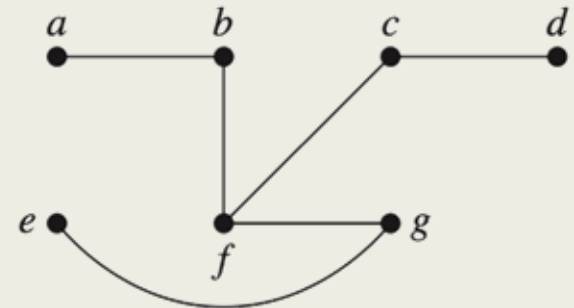
Solution: The graph G is connected, but it is not a tree because it contains simple circuits. Remove the edge $\{a, e\}$. This eliminates one simple circuit, and the resulting subgraph is still connected and still contains every vertex of G . Next remove the edge $\{e, f\}$ to eliminate a second simple circuit. Finally, remove edge $\{c, g\}$ to produce a simple graph with no simple circuits. This subgraph is a spanning tree, because it is a tree that contains every vertex of G .



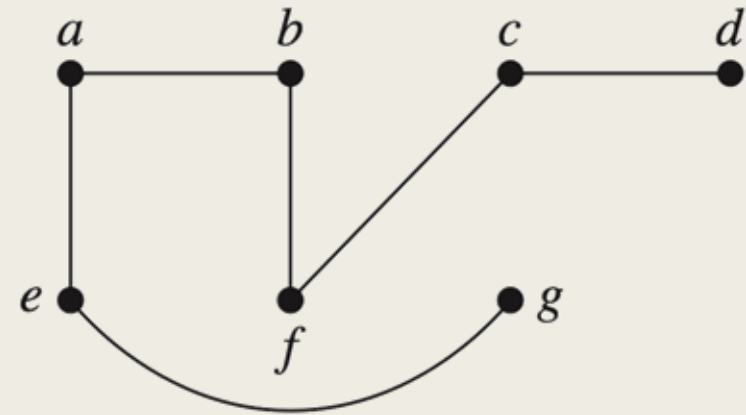
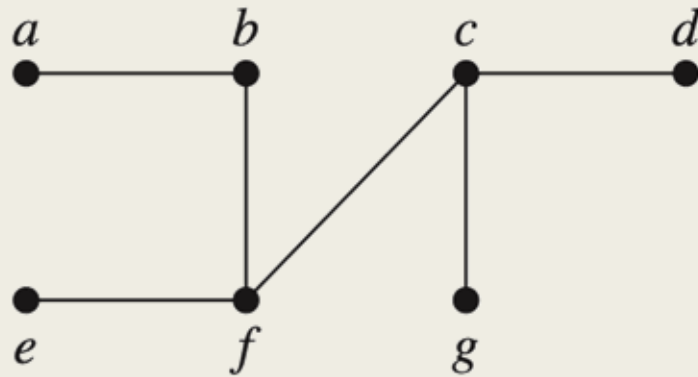
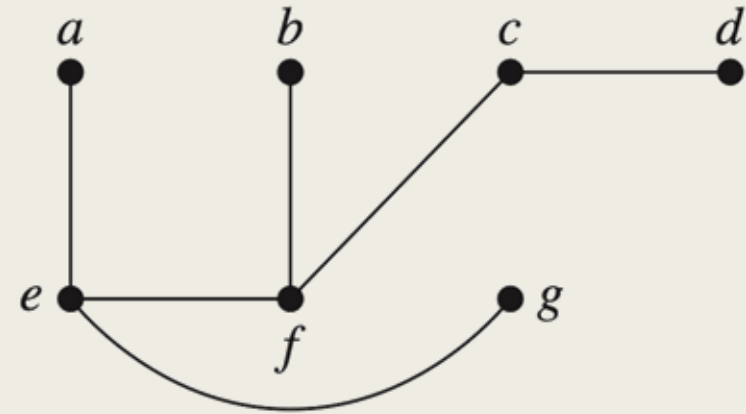
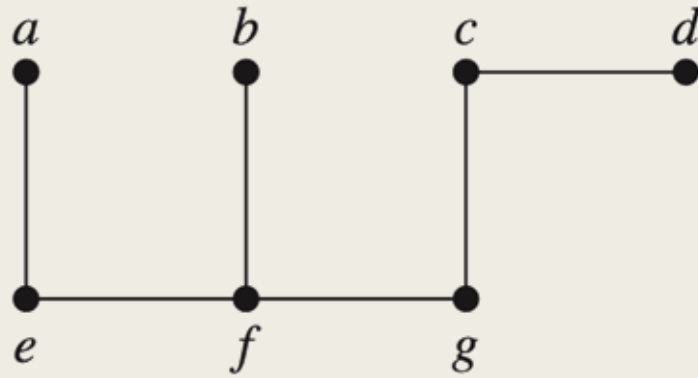
Edge removed: $\{a, e\}$



$\{e, f\}$



$\{c, g\}$



THEOREM. A simple graph is connected if and only if it has a spanning tree.